```
require('./minigame/pixi-adapter/index');
require('./minigame/game-bundle.js');
/**
 * 无尽纹章 - 小游戏入口（须先于 PIXI.Application 加载 unsafe-eval patch）
 */
import '@/core/pixiUnsafeEvalPatch';
import { createPixiHost } from '@/boot/createPixiApp';
import { AssetLoader } from '@/core/AssetLoader';
import { GAME_KEY, GAME_TITLE } from '@/config/gameKey';
import { GameFlow } from '@/view/GameFlow';
import '@/platform/wxPlatform';
declare const GameGlobal: any;
function formatBootErr(e: unknown): string {
  if (e == null) return String(e);
  if (typeof e === 'string') return e;
  if (e instanceof Error) return e.stack || e.message;
  if (typeof e === 'object') {
    const o = e as { errMsg?: unknown; message?: unknown; reason?: unknown };
    if (o.errMsg != null) return String(o.errMsg);
    if (o.message != null) return String(o.message);
    if (o.reason != null) return formatBootErr(o.reason);
    try {
      return JSON.stringify(e);
    } catch {
      return Object.prototype.toString.call(e);
    }
  }
  return String(e);
}
if (typeof GameGlobal !== 'undefined') {
  GameGlobal.onError = (msg: unknown) => {
    console.error('[GlobalError]', formatBootErr(msg));
  };
  GameGlobal.onUnhandledRejection = (ev: unknown) => {
    console.error('[UnhandledRejection]', formatBootErr(ev));
  };
}
function boot(): void {
  const canvas =
    (typeof GameGlobal !== 'undefined' && GameGlobal.canvas) ||
    (typeof window !== 'undefined' && (window as unknown as { canvas?: HTMLCanvasElement }).canva...
    null;
  if (!canvas) {
    console.error('[main] 无法获取 canvas，请确认 pixi-adapter 已加载');
    return;
  }
  const host = createPixiHost(canvas);
  new GameFlow(host);
  try {
    host.renderer.render(host.stage);
```

```
  } catch (e) {
    console.error('[main] 首次 render 失败:', e);
  }
  console.log(`[main] ${GAME_TITLE} (${GAME_KEY}) MVP 启动, screen:`, host.screen.width, 'x', host....
  void AssetLoader.prefetchManifest();
}
/** 微信首帧 canvas 尺寸偶发未就绪：双 rAF 再启动（huahua 类项目常用） */
requestAnimationFrame(() => {
  requestAnimationFrame(() => {
    try {
      boot();
    } catch (e) {
      console.error('[main] boot 异常:', e);
    }
  });
});
/**
 * 现在只导出 GameKey，保证全项目统一引用。
 */
import { GAME_KEY } from '@/config/gameKey';
export { GAME_KEY };
/** 经分 ingest 端点（与 xiao_chu 共用 CloudBase 环境） */
export const ANALYTICS_ENDPOINT =
  'https://rosa-env-d7grf78r5dbd37323.service.tcloudbase.com/analytics-ingest/track';
import type { UnitArchetypeDef, UnitDef, UnitKind, UnitState, Vec2 } from './types';
import { effectiveUnitDef } from './effectiveUnit';
import { cellsFromDist, reachableCells } from './path';
import { manhattan } from './grid';
import { computeDamage, counterMultiplier } from './damage';
import type { TerrainGrid } from './grid';
import { getTerrainAt } from './grid';
import { getTerrainSpec } from '@/data/terrainSpec';
export type AiDifficulty = 'easy' | 'normal' | 'hard';
function key(p: Vec2): string {
  return `${p.x},${p.y}`;
}
function living(units: UnitState[]): UnitState[] {
  return units.filter((u) => u.hp > 0);
}
function defOf(u: UnitState, defs: Record<UnitKind, UnitArchetypeDef>): UnitDef {
  return effectiveUnitDef(u, defs);
}
/**
 * 普攻能否从 `from` 打到格子 `cell`（近战邻格 / 远程 1..range）。
 *
 * 只要 `isRanged` 与 `range` 两个字段，不要求完整 `UnitDef`：威胁染色只有射程信息，
 * 没有也不需要凑出一整个单位定义。
 */
export function canAttackCell(
  atkDef: Pick<UnitDef, 'isRanged' | 'range'>,
```

```
  from: Vec2,
  cell: Vec2,
): boolean {
  const d = manhattan(from, cell);
  if (atkDef.isRanged) return d >= 1 && d <= atkDef.range;
  return d === 1;
}
export function canAttackFrom(
  atkDef: UnitDef,
  fromPos: Vec2,
  target: UnitState,
): boolean {
  return canAttackCell(atkDef, fromPos, target.pos);
}
/**
 * 从 `fromPos` 出发要打谁；射程内无人返回 null。
 *
 * 导出是给引擎的「续打」用的：人工模式按下跳过时，单位可能已经由玩家走到某格了，
 * 这时不能再用 `chooseTurnAction` 的结果--那个目标是配着它想去的**另一格**算出来的，
 * 从玩家选的位置未必打得到，续打就会莫名少一刀。
 */
export function selectAttackTarget(
  atkDef: UnitDef,
  fromPos: Vec2,
  foes: UnitState[],
  defs: Record<UnitKind, UnitArchetypeDef>,
  difficulty: AiDifficulty,
): UnitState | null {
  const alive = living(foes);
  const inRange = alive.filter((t) => canAttackFrom(atkDef, fromPos, t));
  if (inRange.length === 0) return null;
  const taunters = inRange.filter((t) => defOf(t, defs).taunt);
  if (taunters.length > 0) {
    return taunters.reduce((a, b) => (a.hp <= b.hp ? a : b));
  }
  if (difficulty === 'easy') {
    return inRange[Math.floor(Math.random() * inRange.length)]!;
  }
  if (difficulty === 'hard') {
    let bestScore = -Infinity;
    let best: UnitState = inRange[0]!;
    for (const t of inRange) {
      const cm = counterMultiplier(atkDef.id, t.defId);
      const killBonus = t.hp <= atkDef.atk * cm ? 100 : 0;
      const score = cm * 10 + killBonus - t.hp * 0.1;
      if (score > bestScore) {
        bestScore = score;
        best = t;
      }
    }
```

```
    return best;
  }
  return inRange.reduce((a, b) => (a.hp <= b.hp ? a : b));
}
export interface TurnChoice {
  moveTo: Vec2 | null;
  attackTarget: UnitState | null;
}
function evaluateCell(
  cell: Vec2,
  self: UnitState,
  atkDef: UnitDef,
  allUnits: UnitState[],
  defs: Record<UnitKind, UnitArchetypeDef>,
  terrain: TerrainGrid,
  difficulty: AiDifficulty,
): { score: number; target: UnitState | null } {
  const foes = living(allUnits).filter((u) => u.faction !== self.faction);
  const t = selectAttackTarget(atkDef, cell, foes, defs, difficulty);
  const dmg = t ? computeDamage(atkDef, defOf(t, defs), terrain, cell, t.pos) : 0;
  let score = dmg;
  if (difficulty === 'hard') {
    const tSpec = getTerrainSpec(getTerrainAt(terrain, cell));
    score += tSpec.atkMul > 1 ? 5 : 0;
    score += tSpec.defMul < 1 ? 3 : 0;
    score -= tSpec.dotPerRound > 0 ? 4 : 0;
    if (t && t.hp <= dmg) score += 20;
  }
  return { score, target: t };
}
export function chooseTurnAction(
  self: UnitState,
  defs: Record<UnitKind, UnitArchetypeDef>,
  allUnits: UnitState[],
  terrain: TerrainGrid,
  difficulty: AiDifficulty = 'normal',
): TurnChoice {
  if (difficulty === 'easy' && Math.random() < 0.15) {
    return { moveTo: null, attackTarget: null };
  }
  const atkDef = defOf(self, defs);
  const blocked = new Set(
    living(allUnits)
      .filter((u) => u.uid !== self.uid)
      .map((u) => key(u.pos)),
  );
  const dist = reachableCells(self.pos, atkDef.move, blocked, terrain);
  const candidates = cellsFromDist(self.pos, dist);
  let bestScore = -1;
  let bestMoveCost = 99;
```

```
  let bestCell: Vec2 = self.pos;
  let bestTarget: UnitState | null = null;
  for (const cell of candidates) {
    const moveCost = manhattan(cell, self.pos);
    const { score, target } = evaluateCell(cell, self, atkDef, allUnits, defs, terrain, difficulty);
    if (score > bestScore || (score === bestScore && score > 0 && moveCost < bestMoveCost)) {
      bestScore = score;
      bestMoveCost = moveCost;
      bestCell = cell;
      bestTarget = target;
    }
  }
  if (bestScore > 0 && bestTarget) {
    const moveTo = manhattan(bestCell, self.pos) === 0 ? null : bestCell;
    return { moveTo, attackTarget: bestTarget };
  }
  const enemies = living(allUnits).filter((u) => u.faction !== self.faction);
  if (enemies.length === 0) return { moveTo: null, attackTarget: null };
  // 打不到任何人时朝最近的敌人走
  const nearest = enemies.reduce((a, b) =>
    (manhattan(self.pos, a.pos) <= manhattan(self.pos, b.pos) ? a : b));
  let bestDist = Infinity;
  let walk: Vec2 | null = null;
  for (const cell of candidates) {
    if (manhattan(cell, self.pos) === 0) continue;
    const d = manhattan(cell, nearest.pos);
    if (d < bestDist) {
      bestDist = d;
      walk = cell;
    }
  }
  return { moveTo: walk, attackTarget: null };
}
/** 三角克制：攻击方 kind -> 被优先进攻的 kind（若目标为该 kind 则伤害 x） */
export const COUNTER_STRONG = 1.25;
export const COUNTER_WEAK = 0.85;
/** 回合上限：控制单场时长（超时判负），也防止死循环 */
export const MAX_BATTLE_ROUNDS = 30;
/** 玩家部署在战场最下两行（行号从 0 起，随关卡 `terrain` 高度变化） */
export function playerDeployRowRange(gridHeight: number): readonly [number, number] {
  const h = Math.max(2, gridHeight);
  return [h - 2, h - 1] as const;
}
import type { UnitDef, UnitKind, Vec2 } from './types';
import { COUNTER_STRONG, COUNTER_WEAK } from './constants';
import { getTerrainAt, type TerrainGrid } from './grid';
import { getTerrainSpec } from '@/data/terrainSpec';
/** 骑 -> 剑 -> 弓 -> 骑；盾卫不参与 */
export function counterMultiplier(attacker: UnitKind, target: UnitKind): number {
  if (attacker === 'shield' || target === 'shield') return 1;
```

```
  const strong = (
    (attacker === 'cavalry' && target === 'sword')
    || (attacker === 'sword' && target === 'bow')
    || (attacker === 'bow' && target === 'cavalry')
  );
  const weak = (
    (attacker === 'sword' && target === 'cavalry')
    || (attacker === 'bow' && target === 'sword')
    || (attacker === 'cavalry' && target === 'bow')
  );
  if (strong) return COUNTER_STRONG;
  if (weak) return COUNTER_WEAK;
  return 1;
}
export function terrainAttackMul(
  terrainGrid: TerrainGrid,
  attackerPos: Vec2,
): number {
  const tid = getTerrainAt(terrainGrid, attackerPos);
  return getTerrainSpec(tid).atkMul;
}
export function terrainDefenseMul(
  terrainGrid: TerrainGrid,
  targetPos: Vec2,
): number {
  const tid = getTerrainAt(terrainGrid, targetPos);
  return getTerrainSpec(tid).defMul;
}
/**
 * 攻击方所站地形对本次伤害的影响，供回放飘字用；无影响返回 null。
 *
 * 文案从 spec 现算而不是写死：改了 `atkMul` 忘了改文案，玩家看到的数字和飘字就会对不上，
 * 而这类不一致比没有飘字更伤--它教会玩家不要相信 UI。
 */
export function terrainAttackNote(terrainGrid: TerrainGrid, attackerPos: Vec2): string | null {
  const spec = getTerrainSpec(getTerrainAt(terrainGrid, attackerPos));
  if (spec.atkMul === 1) return null;
  return `${spec.name} ${formatPct(spec.atkMul)}`;
}
/** 目标所站地形对本次伤害的影响，供回放飘字用；无影响返回 null */
export function terrainDefenseNote(terrainGrid: TerrainGrid, targetPos: Vec2): string | null {
  const spec = getTerrainSpec(getTerrainAt(terrainGrid, targetPos));
  if (spec.defMul === 1) return null;
  return `${spec.name} ${formatPct(spec.defMul)}`;
}
function formatPct(mul: number): string {
  const pct = Math.round((mul - 1) * 100);
  return pct > 0 ? `+${pct}%` : `${pct}%`;
}
/**
```

```
 * 基础伤害：`atk × 克制 × 攻击方地形 × 目标地形`，下限 1。
 *
 * `targetPos` 现在是必填。它曾经是可选的，于是技能伤害那条路径一直没传，
 * 技能等于无视目标地形--只要有一种可通行地形带减伤，同一个森林里的敌人就会
 * 「普攻打不动、技能照样打满」。地形规则必须对普攻和技能完全一致，
 * 不然「站进森林」这条策略是真是假取决于对手用哪一招，没人教得会。
 */
export function computeDamage(
  attackerDef: UnitDef,
  targetDef: UnitDef,
  terrainGrid: TerrainGrid,
  attackerPos: Vec2,
  targetPos: Vec2,
): number {
  const cm = counterMultiplier(attackerDef.id, targetDef.id);
  const tm = terrainAttackMul(terrainGrid, attackerPos);
  const dm = terrainDefenseMul(terrainGrid, targetPos);
  const raw = attackerDef.atk * cm * tm * dm;
  return Math.max(1, Math.floor(raw));
}
import type { UnitArchetypeDef, UnitDef, UnitKind, UnitState } from './types';
import { defaultSkillId, skillDefForId } from '@/data/skillCatalog';
import {
  sumTimedAtkBonus,
  sumTimedAtkDown,
  sumTimedSpdBonus,
  sumTimedSpdDown,
  timedTauntActive,
} from './timedBattleEffects';
/**
 * 合并兵种 `base`/`strike`、上场覆盖与限时 buff/debuff。
 *
 * **技能来源按阵营分叉**:
 * - 我方：`battleSkill`，没有则回退 `defaultSkillId(defId)`（职业默认技）
 * - 敌方：**只认显式 `battleSkill`**，没有就是纯普攻
 *
 * 敌方曾经也走 defaultSkillId，结果草原黏泥怪会放「旋风斩」、血牙狼自带冲锋被动--
 * 魔物读起来像穿着玩家职业皮的假人，第一章小怪也因此偏强。现在小怪默认无技能，
 * Boss / 高级怪通过关卡蓝图的 `skillSkin` 显式挂上（见 `enemySkillCatalog`）。
 */
export function effectiveUnitDef(
  u: UnitState,
  defs: Record<UnitKind, UnitArchetypeDef>,
): UnitDef {
  const b = defs[u.defId];
  const sk = u.faction === 'enemy'
    ? u.battleSkill
    : (u.battleSkill ?? skillDefForId(defaultSkillId(u.defId)));
  const base = b.base;
  const strike = b.strike;
```

```
  const baseAtk = u.mercAtk ?? base.atk;
  const baseSpd = u.mercSpd ?? base.spd;
  const baseMove = u.mercMove ?? base.move;
  const baseMaxHp = u.mercMaxHp ?? base.maxHp;
  const baseRange = u.mercRange ?? strike.range;
  const baseRanged = u.mercIsRanged ?? strike.isRanged;
  const strikeTaunt = u.mercTaunt ?? strike.taunt;
  const taunt = strikeTaunt || timedTauntActive(u);
  const timedAtk = sumTimedAtkBonus(u);
  const timedAtkDown = sumTimedAtkDown(u);
  const timedSpd = sumTimedSpdBonus(u);
  const timedSpdDown = sumTimedSpdDown(u);
  return {
    id: b.id,
    name: b.name,
    skill: sk,
    tempSkill: u.tempSkill,
    maxHp: baseMaxHp,
    atk: Math.max(1, baseAtk + (u.bonusAtk ?? 0) + timedAtk - timedAtkDown),
    spd: Math.max(1, baseSpd + (u.bonusSpd ?? 0) + timedSpd - timedSpdDown),
    move: Math.max(1, baseMove + (u.bonusMove ?? 0)),
    range: baseRange,
    isRanged: baseRanged,
    taunt,
  };
}
import type {
  BattleEvent,
  BattleReport,
  Faction,
  UnitArchetypeDef,
  UnitKind,
  UnitState,
  Vec2,
} from './types';
import { effectiveUnitDef } from './effectiveUnit';
import { canAttackFrom, chooseTurnAction, selectAttackTarget, type AiDifficulty } from './ai';
import { computeDamage, terrainAttackNote, terrainDefenseNote } from './damage';
import { POTION_DEFS } from '@/data/potionCatalog';
import { getTerrainAt, type TerrainGrid } from './grid';
import { MAX_BATTLE_ROUNDS } from './constants';
import { cellsFromDist, reachableCells, shortestPath4 } from './path';
import {
  castSkillManual,
  skillAiming,
  type SkillSlot,
  trySkillAfterMove,
  trySkillBeforeMove,
  unitSkillSpec,
  type SkillAiming,
```

```
} from './skills';
import { tickTimedBattleEffects } from './timedBattleEffects';
import { getTerrainSpec } from '@/data/terrainSpec';
function key(p: Vec2): string {
 return `${p.x},${p.y}`;
}
function cloneUnits(units: UnitState[]): UnitState[] {
 return units.map((u) => ({
  uid: u.uid,
  defId: u.defId,
  faction: u.faction,
  hp: u.hp,
  pos: { ...u.pos },
  skillCd: u.skillCd ?? 0,
  movedInTurn: false,
  battleSkill: u.battleSkill,
  tempSkill: u.tempSkill,
  tempSkillCd: u.tempSkillCd ?? 0,
  skillMods: u.skillMods ? [...u.skillMods] : undefined,
  bonusAtk: u.bonusAtk,
  bonusSpd: u.bonusSpd,
  bonusMove: u.bonusMove,
  rosterId: u.rosterId,
  displayName: u.displayName,
  boss: u.boss,
  animSet: u.animSet,
  mercMaxHp: u.mercMaxHp,
  mercAtk: u.mercAtk,
  mercSpd: u.mercSpd,
  mercMove: u.mercMove,
  mercRange: u.mercRange,
  mercIsRanged: u.mercIsRanged,
  mercTaunt: u.mercTaunt,
  timedBattleEffects: u.timedBattleEffects?.map((e) => {
   switch (e.kind) {
    case 'taunt':
     return { kind: 'taunt' as const, roundsLeft: e.roundsLeft };
    case 'atkBonus':
     return { kind: 'atkBonus' as const, addAtk: e.addAtk, roundsLeft: e.roundsLeft };
    case 'atkDown':
     return { kind: 'atkDown' as const, subAtk: e.subAtk, roundsLeft: e.roundsLeft };
    case 'spdDown':
     return { kind: 'spdDown' as const, subSpd: e.subSpd, roundsLeft: e.roundsLeft };
    case 'spdBonus':
     return { kind: 'spdBonus' as const, addSpd: e.addSpd, roundsLeft: e.roundsLeft };
    case 'poison':
     return { kind: 'poison' as const, dmgPerRound: e.dmgPerRound, roundsLeft: e.roundsLeft };
   }
  }),
 }));
```

```
}
function buildBlocked(units: UnitState[], selfUid: string): Set<string> {
 const s = new Set<string>();
 for (const u of units) {
  if (u.hp <= 0) continue;
  if (u.uid === selfUid) continue;
  s.add(key(u.pos));
 }
 return s;
}
function bySpeedOrder(units: UnitState[], defs: Record<UnitKind, UnitArchetypeDef>): UnitState[] {
 return [...units]
  .filter((u) => u.hp > 0)
  .sort((a, b) => {
   const sa = effectiveUnitDef(a, defs).spd;
   const sb = effectiveUnitDef(b, defs).spd;
   if (sb !== sa) return sb - sa;
   return a.uid.localeCompare(b.uid);
  });
}
function checkWinner(units: UnitState[]): 'player' | 'enemy' | null {
 const p = units.some((u) => u.faction === 'player' && u.hp > 0);
 const e = units.some((u) => u.faction === 'enemy' && u.hp > 0);
 if (p && e) return null;
 if (p) return 'player';
 return 'enemy';
}
/**
 * `manual`：玩家单位逐个停下来等指令（走位 / 技能 / 普攻全由玩家决定）。
 * `auto`：全部交给程序决策代打，用于扫荡已通过的关卡。
 */
export type BattleMode = 'manual' | 'auto';
export interface BattleSimOptions {
 aiDifficulty?: AiDifficulty;
 /** 缺省 `auto`，保持 `runBattle` 等纯模拟调用方的行为不变 */
 mode?: BattleMode;
 /**
  * 敌人死亡时是否在死亡格掉药剂。只给无尽试炼开：
  * 主线有商店买药，再在战场上掉会让补给点变得可有可无。
  */
 enableDrops?: boolean;
 /** 掉落掷点，测试传入固定序列 */
 dropRng?: () => number;
 dropChance?: number;
}
/**
 * 人工模式下当前单位的内部进度。
 *
 * 一个回合里**移动、技能、普攻各一次**，顺序不限。这里不采用「行动即结束回合」的经典
 * 战棋规则，因为程序决策走的 `actTurn` 是技能和普攻都打的（见 `actTurn`）--限制成二选一，
```

```
 * 人工输出就直接比自动低一截，扫荡会严格优于亲手打，那玩家没有理由认真玩。
 * 规则对齐程序决策之后，人工模式相对自动的优势才落在「走位和时机」这件唯一该由人做的事上。
 *
 * `startPos` 只为撤销移动留着。撤销不需要整份快照：移动只改 `pos` 和 `movedInTurn`
 * （地形每回合掉血在 `startRound` 结算，不在踏入时触发），复位这两个就是精确回滚。
 */
interface MutablePending {
  uid: string;
  moved: boolean;
  usedSkill: boolean;
  attacked: boolean;
  startPos: Vec2;
}
/** 一次 step 的结果：本步产生的事件 + 是否结束 */
export interface BattleStep {
  events: BattleEvent[];
  done: boolean;
  winner: Faction | null;
}
/**
 * 人工模式下，当前行动单位还剩哪些操作没用。
 *
 * `can*` 和 `did*` 都要给：光看 `canSkill: false` 分不清是「这回合已经放过了」
 * 还是「冷却中 / 范围里没目标」。玩家看到一个灰掉的按钮，这两种原因要求的
 * 下一步完全不同（前者是接着打普攻，后者是走位或换人），所以界面必须能区分。
 */
/** 两个技能槽的固定顺序，UI 生成按钮和引擎枚举可放槽位都按它走 */
const SKILL_SLOTS: readonly SkillSlot[] = ['main', 'temp'];
export interface PendingTurn {
  uid: string;
  /** 还能移动（每回合一次；与技能/普攻顺序不限，和程序决策 `actTurn` 一致） */
  canMove: boolean;
  /**
   * 还能放技能（任一槽）。
   *
   * 两个槽**共用**这一个额度：临时技能给的是「主技能在冷却时还有别的事可做」，
   * 不是每回合多打一发。具体哪些槽此刻放得出来看 `castableSlots`。
   */
  canSkill: boolean;
  /** 此刻放得出来的槽（主 / 临时），按钮按它来生成 */
  castableSlots: SkillSlot[];
  /** 还能普攻 */
  canAttack: boolean;
  /** 已经移动过、但还没出手，可以撤销回原位 */
  canUndoMove: boolean;
  didMove: boolean;
  didSkill: boolean;
  didAttack: boolean;
}
/**
```

```
 * 逐步战斗模拟器。
 *
 * 两种推进方式，视图侧按 `pending()` 是否为空来分流：
 * - `stepTurn()`：推进一个回合边界，或让一个 **程序决策单位** 打完整个回合（敌方 / 自动模式）。
 * - `command*()`：人工模式下逐个动作推进当前玩家单位，每个都返回本次产生的事件。
 *
 * 两条路产出的事件格式完全一致，所以回放层只需要一套 `playEvent`。
 */
export interface BattleSim {
  stepTurn(): BattleStep;
  /** 人工模式下正在等指令的单位；null = 该调 `stepTurn()` 了 */
  pending(): PendingTurn | null;
  /** 从当前位置可移动到的格（不含脚下那格） */
  legalMoveCells(uid: string): Vec2[];
  /** 从当前位置能普攻到的敌人 uid */
  legalAttackTargets(uid: string): string[];
  /** 技能瞄准信息；null = 当前放不出来（冷却 / 范围内无目标 / 已用过） */
  skillAiming(uid: string, slot?: SkillSlot): SkillAiming | null;
  /** 移动到指定格（必须在 `legalMoveCells` 内，否则原地不动返回空事件） */
  commandMove(uid: string, cell: Vec2): BattleStep;
  /** 撤销本回合的移动（仅在还没出手时可用） */
  commandUndoMove(uid: string): BattleStep;
  /**
   * 放技能。
   * - 直线/范围确认：`aimCell` 见 `SkillAiming.aimCells`
   * - 无需瞄准：两者都可省略
   */
  commandSkill(uid: string, targetUid?: string, slot?: SkillSlot, aimCell?: Vec2): BattleStep;
  commandAttack(uid: string, targetUid: string): BattleStep;
  /** 结束该单位回合（待机） */
  commandWait(uid: string): BattleStep;
  /** 使用药剂（立即生效，返回产生的事件供回放展示） */
  usePotion(potionId: string): BattleEvent[];
  /**
   * 中途切换托管（自动 / 手动），返回 **切换这一下** 立即产生的事件供回放播出。
   *
   * 切到自动时，正在等指令的那个单位由程序决策接手打完（没用完的动作接着打，见 `actTurn`
   * 的 resume），所以会有事件；切回手动时只是不再代打 **下一个** 单位，事件为空。
   * 已经结算完的动作不回滚--玩家从下一个该他动的单位开始接手。
   */
  setAuto(on: boolean): BattleStep;
  /** 当前是不是程序决策代打玩家单位 */
  isAuto(): boolean;
  /**
   * 跳过：一口气模拟到结束，返回完整战报（含之前已消费的事件）。
   * 人工模式下也可用--剩下的交给程序决策代打，当前单位没用完的动作会被接着打完。
   */
  runToEnd(): BattleReport;
  /** 当前存活单位（实时状态，供 UI 读取血量/冷却） */
  getUnits(): UnitState[];
```

```
  getUnit(uid: string): UnitState | undefined;
  /**
   * 本回合还没动到的单位（uid）。
   * 当前正在行动的人已被弹出，不在这里--UI 要自己把 current 拼到队首。
   */
  roundOrder(): string[];
  /**
   * 出手顺序预览：当前行动者（可选）+ 本回合剩余 + 按速度预估的后续回合，
   * 凑满 `limit` 个。顺序条要靠这条做跨回合规划，不能只看本回合尾巴。
   */
  upcomingOrder(limit: number, currentUid?: string | null): string[];
  getRound(): number;
  isDone(): boolean;
}
export function createBattleSim(
  initialUnits: UnitState[],
  terrain: TerrainGrid,
  defs: Record<UnitKind, UnitArchetypeDef>,
  opts: BattleSimOptions = {},
): BattleSim {
  const units = cloneUnits(initialUnits);
  const aiDifficulty = opts.aiDifficulty ?? 'normal';
  const mode: BattleMode = opts.mode ?? 'auto';
  const enableDrops = opts.enableDrops ?? false;
  const dropRng = opts.dropRng ?? Math.random;
  const dropChance = opts.dropChance ?? 0.35;
  const pickups: { pos: Vec2; potionId: string }[] = [];
  const rolledDeaths = new Set<string>();
  const DROP_POTIONS = ['heal', 'heal', 'heal', 'draught', 'slow'] as const;
  const allEvents: BattleEvent[] = [];
  /**
   * 置位时玩家单位也由程序决策接手（开局选了自动，或中途切了托管 / 按了跳过）。
   *
   * 和 `mode` 分开：`mode` 是这一局的开场状态，`forceAuto` 是当下的实际状态。
   * 合成一个变量的话「托管」就成了单向门--切过去之后没有东西记得原本该由谁操作。
   */
  let forceAuto = mode === 'auto';
  /** 人工模式下当前停下来等指令的单位 */
  let pendingTurn: MutablePending | null = null;
  let rounds = 0;
  let order: string[] = [];
  let done = false;
  let winner: Faction | null = null;
  function attachDrops(events: BattleEvent[]): BattleEvent[] {
    if (!enableDrops) return events;
    const extra: BattleEvent[] = [];
    for (const ev of events) {
      if (ev.type !== 'death') continue;
      if (rolledDeaths.has(ev.uid)) continue;
      rolledDeaths.add(ev.uid);
```

```
    const u = units.find((x) => x.uid === ev.uid);
    if (!u || u.faction !== 'enemy') continue;
    if (pickups.some((p) => p.pos.x === u.pos.x && p.pos.y === u.pos.y)) continue;
    if (dropRng() >= dropChance) continue;
    const potionId = DROP_POTIONS[Math.floor(dropRng() * DROP_POTIONS.length)]!;
    pickups.push({ pos: { ...u.pos }, potionId });
    extra.push({ type: 'drop', pos: { ...u.pos }, potionId });
   }
   return extra.length ? [...events, ...extra] : events;
  }
  function tryPickup(u: UnitState): BattleEvent[] {
   const i = pickups.findIndex((p) => p.pos.x === u.pos.x && p.pos.y === u.pos.y);
   if (i < 0) return [];
   const drop = pickups.splice(i, 1)[0]!;
   return [{ type: 'pickup', uid: u.uid, pos: { ...drop.pos }, potionId: drop.potionId }];
  }
  function finish(w: Faction, events: BattleEvent[]): BattleStep {
   done = true;
   winner = w;
   pendingTurn = null;
   const evs = attachDrops(events);
   evs.push({ type: 'end', winner: w });
   allEvents.push(...evs);
   return { events: evs, done: true, winner: w };
  }
  /** 沿最短路走到 `to`，逐格发 moveStep 供回放做动画 */
  function moveAlong(self: UnitState, to: Vec2): BattleEvent[] {
   const path = shortestPath4(self.pos, to, buildBlocked(units, self.uid), terrain);
   if (!path || path.length <= 1) return [];
   const events: BattleEvent[] = [];
   for (let i = 1; i < path.length; i++) {
    const from = { ...self.pos };
    const next = { ...path[i]! };
    self.pos = next;
    self.movedInTurn = true;
    events.push({ type: 'moveStep', uid: self.uid, from, to: next });
   }
   return events;
  }
  function liveUnit(uid: string): UnitState | null {
   const u = units.find((x) => x.uid === uid);
   return u && u.hp > 0 ? u : null;
  }
  function reachFrom(u: UnitState): Vec2[] {
   const atkDef = effectiveUnitDef(u, defs);
   const dist = reachableCells(u.pos, atkDef.move, buildBlocked(units, u.uid), terrain);
   return cellsFromDist(u.pos, dist).filter((c) => !(c.x === u.pos.x && c.y === u.pos.y));
  }
  function attackTargetsFrom(u: UnitState): string[] {
   const atkDef = effectiveUnitDef(u, defs);
```

```
  return units
    .filter((t) => t.hp > 0 && t.faction !== u.faction && canAttackFrom(atkDef, u.pos, t))
    .map((t) => t.uid);
}
function startRound(): BattleStep {
  rounds += 1;
  const events: BattleEvent[] = [{ type: 'round', round: rounds }];
  for (const t of tickTimedBattleEffects(units)) {
    events.push({
      type: 'dot',
      uid: t.uid,
      damage: t.damage,
      hpLeft: t.hpLeft,
      source: 'poison',
    });
    if (t.died) events.push({ type: 'death', uid: t.uid });
  }
  for (const u of units) {
    if (u.hp <= 0) continue;
    u.movedInTurn = false;
    if (u.skillCd > 0) u.skillCd -= 1;
    if ((u.tempSkillCd ?? 0) > 0) u.tempSkillCd = (u.tempSkillCd ?? 0) - 1;
    const tSpec = getTerrainSpec(getTerrainAt(terrain, u.pos));
    if (tSpec.dotPerRound > 0) {
      u.hp -= tSpec.dotPerRound;
      events.push({
        type: 'dot',
        uid: u.uid,
        damage: tSpec.dotPerRound,
        hpLeft: Math.max(0, u.hp),
        source: 'terrain',
      });
      if (u.hp <= 0) {
        events.push({ type: 'death', uid: u.uid });
      }
    }
  }
  order = bySpeedOrder(units, defs).map((u) => u.uid);
  const evs = attachDrops(events);
  const w = checkWinner(units);
  if (w) return finish(w, evs);
  allEvents.push(...evs);
  return { events: evs, done: false, winner: null };
}
/**
 * 程序决策打完一个单位的整个回合。
 *
 * `resume` 是人工模式下按了跳过时传进来的进度：该单位可能已经走过、或已经放过技能。
 * 不传这个就只能二选一--要么让程序决策从头再走一遍（同一回合移动两次），要么直接丢掉
 * 它剩下的动作。后者在按跳过的那一刻会白送一次攻击机会，而玩家通常正是因为
```

```
 * 局势已定才按跳过的，凭空掉一刀会让结果和他的判断不符。
 */
function actTurn(self: UnitState, resume?: MutablePending): BattleStep {
  const events: BattleEvent[] = [];
  const canSkill = !resume?.usedSkill;
  const canMove = !resume?.moved;
  const canAttack = !resume?.attacked;
  if (resume?.moved) self.movedInTurn = true;
  if (canSkill) {
    events.push(...trySkillBeforeMove(self, defs, units, terrain));
  }
  let w = checkWinner(units);
  if (w) return finish(w, events);
  const choice = chooseTurnAction(
    self, defs, units, terrain,
    self.faction === 'enemy' ? aiDifficulty : 'normal',
  );
  const atkDef = effectiveUnitDef(self, defs);
  if (canMove) {
    const blockedReach = buildBlocked(units, self.uid);
    const reachDist = reachableCells(self.pos, atkDef.move, blockedReach, terrain);
    events.push({
      type: 'moveRange',
      uid: self.uid,
      cells: cellsFromDist(self.pos, reachDist),
    });
  }
  if (canMove && choice.moveTo) {
    events.push(...moveAlong(self, choice.moveTo));
  }
  if (canSkill) {
    events.push(...trySkillAfterMove(self, defs, units, terrain));
  }
  w = checkWinner(units);
  if (w) return finish(w, events);
  // 已经由玩家走过位了，就不能沿用 choice 里那个「配着别的落点」算出来的目标
  const tgt = !canAttack
    ? null
    : canMove
      ? choice.attackTarget
      : selectAttackTarget(
        atkDef,
        self.pos,
        units.filter((u) => u.faction !== self.faction),
        defs,
        self.faction === 'enemy' ? aiDifficulty : 'normal',
      );
  if (tgt && tgt.hp > 0) {
    const tLive = units.find((u) => u.uid === tgt.uid);
    // 目标可能已被技能击杀，或因位移脱战
```

```
    if (tLive && tLive.hp > 0 && canAttackFrom(atkDef, self.pos, tLive)) {
      events.push(...basicAttack(self, tLive));
    }
  }
  w = checkWinner(units);
  if (w) return finish(w, events);
  const evs = attachDrops(events);
  allEvents.push(...evs);
  return { events: evs, done: false, winner: null };
}
/**
 * 一次普攻的结算。程序决策回合和玩家指令共用，不要各写一份--
 * 冲锋加成、地形归因这些散在两条路上迟早会分叉，而分叉的表现是
 * 「同一招我打出 12、电脑打出 15」，玩家只会得出数值不可信的结论。
 */
function basicAttack(self: UnitState, target: UnitState): BattleEvent[] {
  const atkDef = effectiveUnitDef(self, defs);
  const defT = effectiveUnitDef(target, defs);
  let dmg = computeDamage(atkDef, defT, terrain, self.pos, target.pos);
  const sk = atkDef.skill;
  // 走 `unitSkillSpec` 而不是 `getSkillSpec`：冲锋的倍率也吃词条（「蓄势」「践地」），
  // 读原始规格的话那两条选了等于没选，而表现只是普攻数字偏小，肉眼查不出来。
  const chargeMul = sk ? unitSkillSpec(self, sk.id)?.passiveBasicAttackMulIfMoved : undefined;
  const charged = Boolean(chargeMul) && self.movedInTurn;
  if (chargeMul && charged) {
    dmg = Math.max(1, Math.floor(dmg * chargeMul));
  }
  target.hp -= dmg;
  const events: BattleEvent[] = [{
    type: 'attack',
    attacker: self.uid,
    target: target.uid,
    damage: dmg,
    hpLeft: Math.max(0, target.hp),
    // 冲锋和地形一样，改了这一下的伤害就得说出来。骑兵的全部身份就是「先动再打更疼」，
    // 而在此之前它只体现为一个玩家无从比较的数字，等于这个被动不存在。
    attackLabel: charged ? '冲锋' : '普攻',
    charged: charged || undefined,
    // 地形改了这一下的伤害就说出来。地形之前「没用」很大一部分是因为它从不出声：
    // 高地 +25% 只体现为一个玩家无从比较的数字。
    atkTerrainNote: terrainAttackNote(terrain, self.pos) ?? undefined,
    defTerrainNote: terrainDefenseNote(terrain, target.pos) ?? undefined,
  }];
  if (target.hp <= 0) events.push({ type: 'death', uid: target.uid });
  return events;
}
function stepTurn(): BattleStep {
  if (done) return { events: [], done: true, winner };
  // 还有人在等指令时不许推进，否则玩家那个单位会被跳过
  if (pendingTurn) return { events: [], done: false, winner: null };
```

```
  // 回合边界
  if (order.length === 0) {
    const w0 = checkWinner(units);
    if (w0) return finish(w0, []);
    if (rounds >= MAX_BATTLE_ROUNDS) return finish('enemy', []);
    return startRound();
  }
  // 弹出下一个存活行动者
  while (order.length > 0) {
    const uid = order.shift()!;
    const self = units.find((u) => u.uid === uid);
    if (!self || self.hp <= 0) continue;
    const turnStart: BattleEvent = { type: 'turnStart', uid: self.uid, faction: self.faction };
    if (self.faction === 'player' && !forceAuto) {
      pendingTurn = {
        uid: self.uid,
        moved: false,
        usedSkill: false,
        attacked: false,
        startPos: { ...self.pos },
      };
      allEvents.push(turnStart);
      return { events: [turnStart], done: false, winner: null };
    }
    const step = actTurn(self);
    return { ...step, events: [turnStart, ...step.events] };
  }
  // 本回合行动者全部阵亡 -> 直接进入下一回合边界
  return stepTurn();
}
// ============ 人工模式：逐指令推进 ============
/** 当前等指令的单位；顺带处理它已经死了的情况（被反击/地形打死不会发生，但别留坑） */
function activePending(uid: string): { p: MutablePending; u: UnitState } | null {
  if (done || !pendingTurn || pendingTurn.uid !== uid) return null;
  const u = liveUnit(uid);
  if (!u) return null;
  return { p: pendingTurn, u };
}
function pendingView(): PendingTurn | null {
  if (!pendingTurn) return null;
  const u = liveUnit(pendingTurn.uid);
  if (!u) return null;
  const acted = pendingTurn.usedSkill || pendingTurn.attacked;
  const castable = castableSlotsFor(u);
  return {
    uid: pendingTurn.uid,
    // 移动与出手顺序不限：程序决策是先技能再走位再普攻的（见 `actTurn`），
    // 人工若「先放技能就锁死移动」，玩家永远学不会那套起手，也会白白丢掉走位。
    // 撤招只在「走过且还没出手」时开放--出手后再撤等于改写已经结算的伤害。
    canMove: !pendingTurn.moved,
```

```
    canSkill: castable.length > 0,
    castableSlots: castable,
    canAttack: !pendingTurn.attacked && attackTargetsFrom(u).length > 0,
    canUndoMove: pendingTurn.moved && !acted,
    didMove: pendingTurn.moved,
    didSkill: pendingTurn.usedSkill,
    didAttack: pendingTurn.attacked,
  };
}
function skillAimingFor(u: UnitState, slot: SkillSlot = 'main'): SkillAiming | null {
  if (pendingTurn?.uid === u.uid && pendingTurn.usedSkill) return null;
  return skillAiming(u, defs, units, terrain, slot);
}
function castableSlotsFor(u: UnitState): SkillSlot[] {
  return SKILL_SLOTS.filter((slot) => skillAimingFor(u, slot) !== null);
}
/**
 * 一个动作结算完之后：该结束回合就结束。
 *
 * 「无事可做就自动收尾」省掉了绝大多数回合里那一下多余的「结束」点击——
 * 常见回合就是走一步、砍一刀，技能在冷却，此时再要求玩家确认一次纯属噪音。
 * 但只要还有技能能放，就一定停下来等：替玩家决定放弃一次技能，比多一次点击糟得多。
 */
function settleAfterAction(events: BattleEvent[]): BattleStep {
  const evs = attachDrops(events);
  const w = checkWinner(units);
  if (w) return finish(w, evs);
  const v = pendingView();
  // 还有移动/出手额度，或还能撤销走位时，都停下来等。
  // 走完若立刻收尾，玩家来不及点撤销；先技能后走位也不能出手完就掐掉移动。
  if (!v || (!v.canMove && !v.canSkill && !v.canAttack && !v.canUndoMove)) {
    pendingTurn = null;
  }
  allEvents.push(...evs);
  return { events: evs, done: false, winner: null };
}
function noop(): BattleStep {
  return { events: [], done: false, winner: null };
}
function commandMove(uid: string, cell: Vec2): BattleStep {
  const cur = activePending(uid);
  if (!cur) return noop();
  const v = pendingView();
  if (!v?.canMove) return noop();
  if (!reachFrom(cur.u).some((c) => c.x === cell.x && c.y === cell.y)) return noop();
  const events = moveAlong(cur.u, cell);
  if (events.length === 0) return noop();
  cur.p.moved = true;
  // 走完若技能/普攻也没了，settle 会自动收尾；还有出手额度则继续等
  return settleAfterAction(events);
```

```
  }
  function commandUndoMove(uid: string): BattleStep {
    const cur = activePending(uid);
    if (!cur) return noop();
    const v = pendingView();
    if (!v?.canUndoMove) return noop();
    const from = { ...cur.u.pos };
    cur.u.pos = { ...cur.p.startPos };
    cur.u.movedInTurn = false;
    cur.p.moved = false;
    // 用一步 moveStep 直接跳回原位，回放层按普通移动播即可
    const events: BattleEvent[] = [
      { type: 'moveStep', uid, from, to: { ...cur.p.startPos } },
    ];
    allEvents.push(...events);
    return { events, done: false, winner: null };
  }
  function commandSkill(
    uid: string,
    targetUid?: string,
    slot: SkillSlot = 'main',
    aimCell?: Vec2,
  ): BattleStep {
    const cur = activePending(uid);
    if (!cur) return noop();
    const v = pendingView();
    if (!v?.castableSlots.includes(slot)) return noop();
    const events = castSkillManual(cur.u, defs, units, terrain, targetUid, slot, aimCell);
    if (events.length === 0) return noop();
    cur.p.usedSkill = true;
    return settleAfterAction(events);
  }
  function commandAttack(uid: string, targetUid: string): BattleStep {
    const cur = activePending(uid);
    if (!cur) return noop();
    const v = pendingView();
    if (!v?.canAttack) return noop();
    if (!attackTargetsFrom(cur.u).includes(targetUid)) return noop();
    const target = liveUnit(targetUid);
    if (!target) return noop();
    cur.p.attacked = true;
    return settleAfterAction(basicAttack(cur.u, target));
  }
  function commandWait(uid: string): BattleStep {
    const cur = activePending(uid);
    if (!cur) return noop();
    // 走到掉落格上再待机才捡起来：只路过不捡，否则走位会被「顺手拾取」绑死
    const events = tryPickup(cur.u);
    pendingTurn = null;
    allEvents.push(...events);
```

```
  return { events, done: false, winner: null };
 }
 function usePotion(potionId: string): BattleEvent[] {
  if (done) return [];
  const def = POTION_DEFS[potionId];
  if (!def) return [];
  const events: BattleEvent[] = [{ type: 'potion', potionId, name: def.name }];
  const eff = def.effect;
  if (eff.kind === 'healAllies') {
   for (const u of units) {
    if (u.faction !== 'player' || u.hp <= 0) continue;
    const maxHp = effectiveUnitDef(u, defs).maxHp;
    const amount = Math.max(1, Math.floor(maxHp * eff.ratio));
    const healed = Math.min(maxHp, u.hp + amount) - u.hp;
    if (healed <= 0) continue;
    u.hp += healed;
    events.push({ type: 'heal', target: u.uid, amount: healed, hpLeft: u.hp });
   }
  } else if (eff.kind === 'atkBuffAllies') {
   for (const u of units) {
    if (u.faction !== 'player' || u.hp <= 0) continue;
    const baseAtk = effectiveUnitDef(u, defs).atk;
    const addAtk = Math.max(1, Math.floor(baseAtk * eff.atkRatio));
    const list = u.timedBattleEffects ?? [];
    // +1：tick 在回合开始统一 -1，保证 buff 覆盖 N 个完整回合
    list.push({ kind: 'atkBonus', addAtk, roundsLeft: eff.rounds + 1 });
    u.timedBattleEffects = list;
    events.push({ type: 'statusNote', target: u.uid, text: `攻+${addAtk}`, tone: 'buff' });
   }
  } else if (eff.kind === 'slowFoes') {
   for (const u of units) {
    if (u.faction !== 'enemy' || u.hp <= 0) continue;
    const list: typeof u.timedBattleEffects =
     (u.timedBattleEffects ?? []).filter((x) => x.kind !== 'spdDown');
    list.push({ kind: 'spdDown', subSpd: eff.subSpd, roundsLeft: eff.rounds + 1 });
    u.timedBattleEffects = list;
    events.push({
     type: 'statusNote',
     target: u.uid,
     text: `速-${eff.subSpd}`,
     tone: 'debuff',
    });
   }
  }
  allEvents.push(...events);
  return events;
 }
 function setAuto(on: boolean): BattleStep {
  forceAuto = on;
  const idle: BattleStep = { events: [], done, winner };
```

```
    // 切回手动不需要动当前状态：下一次 stepTurn 自然会停下来等指令。
    if (!on || done || !pendingTurn) return idle;
    // 当前单位如果已经走了一半，把剩下的动作接着打完（见 actTurn 的 resume）
    const resume = pendingTurn;
    pendingTurn = null;
    const u = liveUnit(resume.uid);
    return u ? actTurn(u, resume) : idle;
  }
  function runToEnd(): BattleReport {
    setAuto(true);
    while (!done) stepTurn();
    return { events: allEvents, winner: winner ?? 'enemy', rounds };
  }
  return {
    stepTurn,
    pending: pendingView,
    legalMoveCells: (uid) => {
      const u = liveUnit(uid);
      return u ? reachFrom(u) : [];
    },
    legalAttackTargets: (uid) => {
      const u = liveUnit(uid);
      return u ? attackTargetsFrom(u) : [];
    },
    skillAiming: (uid, slot) => {
      const u = liveUnit(uid);
      return u ? skillAimingFor(u, slot ?? 'main') : null;
    },
    commandMove,
    commandUndoMove,
    commandSkill,
    commandAttack,
    commandWait,
    usePotion,
    setAuto,
    isAuto: () => forceAuto,
    runToEnd,
    getUnits: () => units,
    getUnit: (uid) => units.find((u) => u.uid === uid),
    roundOrder: () => [...order],
    upcomingOrder: (limit, currentUid) => {
      const alive = units.filter((u) => u.hp > 0);
      const aliveIds = new Set(alive.map((u) => u.uid));
      const out: string[] = [];
      if (currentUid && aliveIds.has(currentUid)) out.push(currentUid);
      for (const uid of order) {
        if (out.length >= limit) break;
        if (aliveIds.has(uid) && uid !== currentUid) out.push(uid);
      }
      // 本回合队列耗尽后，用当前速度序循环预估下一回合、下下回合……
```

```
    while (out.length < limit && alive.length > 0) {
      const next = bySpeedOrder(alive, defs).map((u) => u.uid);
      let added = 0;
      for (const uid of next) {
        if (out.length >= limit) break;
        out.push(uid);
        added++;
      }
      if (added === 0) break;
    }
    return out;
  },
  getRound: () => rounds,
  isDone: () => done,
 };
}
/** 一次性模拟到结束（全自动）；「跳过回放」与单测复用 */
export function runBattle(
 initialUnits: UnitState[],
 terrain: TerrainGrid,
 defs: Record<UnitKind, UnitArchetypeDef>,
 aiDifficulty: AiDifficulty = 'normal',
): BattleReport {
 const sim = createBattleSim(initialUnits, terrain, defs, { aiDifficulty, mode: 'auto' });
 return sim.runToEnd();
}
import type { TerrainId, Vec2 } from './types';
export type TerrainGrid = TerrainId[][];
export function gridSize(g: TerrainGrid): { w: number; h: number } {
 const h = g.length;
 const w = h > 0 ? (g[0]?.length ?? 0) : 0;
 return { w, h };
}
export function inBounds(p: Vec2, g: TerrainGrid): boolean {
 const { w, h } = gridSize(g);
 if (w <= 0 || h <= 0) return false;
 return p.x >= 0 && p.x < w && p.y >= 0 && p.y < h;
}
export function manhattan(a: Vec2, b: Vec2): number {
 return Math.abs(a.x - b.x) + Math.abs(a.y - b.y);
}
export function getTerrainAt(grid: TerrainGrid, p: Vec2): TerrainId {
 if (!inBounds(p, grid)) return 'plain';
 return grid[p.y]![p.x] ?? 'plain';
}
/** 4 邻格（受当前关卡地形图尺寸约束） */
export function neighbors4(p: Vec2, g: TerrainGrid): Vec2[] {
 return [
  { x: p.x + 1, y: p.y },
  { x: p.x - 1, y: p.y },
```

```
    { x: p.x, y: p.y + 1 },
    { x: p.x, y: p.y - 1 },
  ].filter((n) => inBounds(n, g));
}
/** 生成 `width × height` 全平原地形（每关棋盘尺寸由 `terrain` 矩阵决定） */
export function emptyTerrain(width: number, height: number): TerrainGrid {
  return Array.from({ length: height }, () =>
    Array.from({ length: width }, (): TerrainId => 'plain'),
  );
}
/** 将玩家布阵阶段放置的地形叠到关卡底图上（浅拷贝行） */
export function mergeTerrainOverlay(base: TerrainGrid, overlay: { x: number; y: number; terrain: ...
  const g = base.map((row) => [...row]);
  for (const c of overlay) {
    if (g[c.y]?.[c.x] !== undefined) g[c.y]![c.x] = c.terrain;
  }
  return g;
}
export type {
  BattleEvent,
  BattleReport,
  Faction,
  TerrainId,
  TimedBattleEffect,
  UnitDef,
  UnitKind,
  UnitState,
  Vec2,
} from './types';
export { MAX_BATTLE_ROUNDS, playerDeployRowRange, COUNTER_STRONG, COUNTER_WEAK } from './constants';
export { runBattle } from './engine';
export { effectiveUnitDef } from './effectiveUnit';
export type { SkillDamageContext } from './skillDamage';
export {
  computeSkillHitDamage,
  computeSkillHitDamageWithSpec,
  registerSkillDamageCalculator,
  unregisterSkillDamageCalculator,
} from './skillDamage';
export type { SkillDamageSpec } from '@/data/skillCatalog';
export { emptyTerrain, gridSize, inBounds, mergeTerrainOverlay, neighbors4 } from './grid';
export type { TerrainGrid } from './grid';
import type { TerrainGrid } from './grid';
import { getTerrainAt, neighbors4 } from './grid';
import type { Vec2 } from './types';
import { getTerrainSpec } from '@/data/terrainSpec';
function key(p: Vec2): string {
  return `${p.x},${p.y}`;
}
/**
```

```
 * BFS with per-cell movement cost from TerrainSpec.
 * Impassable terrain (moveCost === Infinity) is never entered.
 */
export function reachableCells(
  start: Vec2,
  moveBudget: number,
  blocked: Set<string>,
  terrain: TerrainGrid,
): Map<string, number> {
  const dist = new Map<string, number>();
  const q: Vec2[] = [start];
  dist.set(key(start), 0);
  let qi = 0;
  while (qi < q.length) {
    const p = q[qi++]!;
    const d = dist.get(key(p))!;
    for (const n of neighbors4(p, terrain)) {
      const nk = key(n);
      if (nk === key(start)) continue;
      if (blocked.has(nk)) continue;
      const tSpec = getTerrainSpec(getTerrainAt(terrain, n));
      if (tSpec.moveCost >= Infinity) continue;
      const nd = d + tSpec.moveCost;
      if (nd > moveBudget) continue;
      if (!dist.has(nk) || nd < dist.get(nk)!) {
        dist.set(nk, nd);
        q.push(n);
      }
    }
  }
  return dist;
}
/** All reachable cells (from BFS dist map) as Vec2[]. */
export function cellsFromDist(_unusedStart: Vec2, dist: Map<string, number>): Vec2[] {
  const out: Vec2[] = [];
  for (const [k] of dist) {
    const [xs, ys] = k.split(',');
    out.push({ x: Number(xs), y: Number(ys) });
  }
  return out;
}
/** Shortest path (4-dir) respecting terrain movement cost. */
export function shortestPath4(
  from: Vec2,
  to: Vec2,
  blocked: Set<string>,
  terrain: TerrainGrid,
): Vec2[] | null {
  const fk = key(from);
  const tk = key(to);
```

```
  if (fk === tk) return [from];
  if (blocked.has(tk)) return null;
  const tDest = getTerrainSpec(getTerrainAt(terrain, to));
  if (tDest.moveCost >= Infinity) return null;
  const dist = new Map<string, number>();
  const parent = new Map<string, string>();
  const q: Vec2[] = [from];
  dist.set(fk, 0);
  parent.set(fk, '');
  let qi = 0;
  while (qi < q.length) {
   const p = q[qi++]!;
   const pk = key(p);
   if (pk === tk) {
    const out: Vec2[] = [];
    let cur: string | undefined = tk;
    while (cur && cur !== fk) {
     const [xa, ya] = cur.split(',');
     out.push({ x: Number(xa), y: Number(ya) });
     cur = parent.get(cur);
    }
    out.push({ ...from });
    out.reverse();
    return out;
   }
   const d = dist.get(pk)!;
   for (const n of neighbors4(p, terrain)) {
    const nk = key(n);
    if (blocked.has(nk)) continue;
    const tSpec = getTerrainSpec(getTerrainAt(terrain, n));
    if (tSpec.moveCost >= Infinity) continue;
    const nd = d + tSpec.moveCost;
    if (!dist.has(nk) || nd < dist.get(nk)!) {
     dist.set(nk, nd);
     parent.set(nk, pk);
     q.push(n);
    }
   }
  }
  return null;
}
import type { SkillDamageSpec, SkillSpec } from '@/data/skillCatalog';
import { computeDamage, counterMultiplier, terrainAttackMul, terrainDefenseMul } from '../damage';
import type { UnitDef } from '../types';
import type { SkillDamageContext } from './context';
function atkDefScaled(base: UnitDef, atk: number): UnitDef {
 return { ...base, atk };
}
function clampDamage(n: number): number {
 return Math.max(1, Math.floor(n));
```

```
}
/** `custom` 用：`registerSkillDamageCalculator(id, fn)` */
const customCalculators = new Map<
  string,
  (ctx: SkillDamageContext, params: Record<string, number> | undefined) => number
>();
/**
 * 注册自定义伤害公式（`damage: { kind: 'custom', id, params }`）。
 * `fn` 返回整数伤害，最终统一钳到 ≥1。
 */
export function registerSkillDamageCalculator(
  id: string,
  fn: (ctx: SkillDamageContext, params: Record<string, number> | undefined) => number,
): void {
  customCalculators.set(id, fn);
}
export function unregisterSkillDamageCalculator(id: string): void {
  customCalculators.delete(id);
}
function scaledStandard(ctx: SkillDamageContext, atkMul: number): number {
  if (atkMul <= 0) return 0;
  const atk = Math.max(1, Math.floor(ctx.casterDef.atk * atkMul));
  const sad = atkDefScaled(ctx.casterDef, atk);
  return clampDamage(computeDamage(sad, ctx.targetDef, ctx.terrain, ctx.self.pos, ctx.target.pos));
}
/**
 * `applyTerrain` 同时管**两头**的地形：施法者站的格（攻击加成）和目标站的格（减伤）。
 * 只吃一头会让「站进森林」对普攻有效、对技能无效，玩家学不会这条规则。
 */
function terrainMul(ctx: SkillDamageContext): number {
  return terrainAttackMul(ctx.terrain, ctx.self.pos) * terrainDefenseMul(ctx.terrain, ctx.target....
}
function flatDamage(
  ctx: SkillDamageContext,
  amount: number,
  applyCounter: boolean,
  applyTerrain: boolean,
): number {
  let n = amount;
  if (applyCounter) n *= counterMultiplier(ctx.casterDef.id, ctx.targetDef.id);
  if (applyTerrain) n *= terrainMul(ctx);
  return clampDamage(n);
}
function percentTargetMaxHp(
  ctx: SkillDamageContext,
  ratio: number,
  applyCounter: boolean,
  applyTerrain: boolean,
): number {
  let n = ctx.targetDef.maxHp * ratio;
```

```
  if (applyCounter) n *= counterMultiplier(ctx.casterDef.id, ctx.targetDef.id);
  if (applyTerrain) n *= terrainMul(ctx);
  return clampDamage(n);
}
/**
 * 「处决」类词条的倍率：目标残血时才生效。
 *
 * 读 `target.hp` 而不是词条侧预判，因为同一次 AoE 里前面的目标已经掉过血了--
 * 谁进了处决线只有结算这一刻知道。
 */
function executeMul(ctx: SkillDamageContext): number {
  const ex = ctx.spec.executeBonus;
  if (!ex) return 1;
  return isExecuting(ctx.spec, ctx.target.hp, ctx.targetDef.maxHp) ? ex.mul : 1;
}
/**
 * 这一击有没有踩到处决线。结算和飘字问的是同一个函数，两边算法分家的话
 * 会出现「飘了处决但伤害没涨」这种查不出来的对不上。
 *
 * 必须在扣血**之前**问：处决读的是命中那一刻的血量。
 */
export function isExecuting(spec: SkillSpec, targetHp: number, targetMaxHp: number): boolean {
  const ex = spec.executeBonus;
  if (!ex || targetMaxHp <= 0) return false;
  return targetHp / targetMaxHp < ex.belowHpRatio;
}
/**
 * 对单个目标结算技能伤害（不含治疗等；`none` 为 0）。
 * 内置种类在 `SkillDamageSpec`；扩展用 `custom` + `registerSkillDamageCalculator`。
 */
export function computeSkillHitDamage(ctx: SkillDamageContext): number {
  const base = dispatchDamage(ctx.spec.damage, ctx);
  if (base <= 0) return base;
  const mul = executeMul(ctx);
  return mul === 1 ? base : clampDamage(base * mul);
}
function dispatchDamage(d: SkillDamageSpec, ctx: SkillDamageContext): number {
  switch (d.kind) {
    case 'none':
      return 0;
    case 'scaledAtk':
      return scaledStandard(ctx, d.atkMul);
    case 'flat':
      return flatDamage(ctx, d.amount, d.applyCounter !== false, d.applyTerrain !== false);
    case 'percentTargetMaxHp':
      return percentTargetMaxHp(ctx, d.ratio, d.applyCounter !== false, d.applyTerrain !== false);
    case 'custom': {
      const fn = customCalculators.get(d.id);
      if (!fn) {
        throw new Error(`[skillDamage] unknown custom calculator id: "${d.id}" (skill ${ctx.spec....
```

```
      }
      return clampDamage(fn(ctx, d.params));
    }
  }
}
/** 供测试或工具：对上下文用另一套 `damage` 试算 */
export function computeSkillHitDamageWithSpec(
  ctx: SkillDamageContext,
  damage: SkillDamageSpec,
): number {
  return dispatchDamage(damage, ctx);
}
import type { UnitArchetypeDef, UnitDef, UnitKind, UnitState } from '../types';
import type { SkillSpec } from '@/data/skillCatalog';
import type { TerrainGrid } from '../grid';
/** 计算「对某一目标」的技能伤害时传入的上下文（多目标技能会多次调用） */
export interface SkillDamageContext {
  self: UnitState;
  target: UnitState;
  /** 施法者本场合并面板 `effectiveUnitDef(self)` */
  casterDef: UnitDef;
  /** 目标本场合并面板 */
  targetDef: UnitDef;
  spec: SkillSpec;
  terrain: TerrainGrid;
  defs: Record<UnitKind, UnitArchetypeDef>;
}
export type { SkillDamageContext } from './context';
export {
  computeSkillHitDamage,
  computeSkillHitDamageWithSpec,
  isExecuting,
  registerSkillDamageCalculator,
  unregisterSkillDamageCalculator,
} from './computeSkillHitDamage';
import type {
  BattleEvent,
  SkillDef,
  SkillHit,
  UnitArchetypeDef,
  UnitDef,
  UnitKind,
  UnitState,
  Vec2,
} from './types';
import { effectiveUnitDef } from './effectiveUnit';
import { terrainAttackNote, terrainDefenseNote } from './damage';
import { computeSkillHitDamage, isExecuting } from './skillDamage';
import {
  applySkillCastAllyEffects,
```

```
  applySkillCastFoeEffects,
  applySkillCastSelfEffects,
} from './timedBattleEffects';
import { canProfessionEquipSkill, getSkillSpec, type SkillSpec } from '@/data/skillCatalog';
import { effectiveSkillSpec } from '@/data/skillModCatalog';
import { gridSize, inBounds, manhattan, neighbors4, type TerrainGrid } from './grid';
const RAY_DIRS: Vec2[] = [
  { x: 1, y: 0 },
  { x: -1, y: 0 },
  { x: 0, y: 1 },
  { x: 0, y: -1 },
];
function livingFoes(self: UnitState, units: UnitState[]): UnitState[] {
  return units.filter((u) => u.hp > 0 && u.faction !== self.faction);
}
function livingAllies(self: UnitState, units: UnitState[]): UnitState[] {
  return units.filter((u) => u.hp > 0 && u.faction === self.faction);
}
function pushDeathIfNeeded(events: BattleEvent[], t: UnitState): void {
  if (t.hp <= 0) events.push({ type: 'death', uid: t.uid });
}
function enemiesOnRay(
  self: UnitState,
  from: Vec2,
  dir: Vec2,
  units: UnitState[],
  terrain: TerrainGrid,
): UnitState[] {
  const out: UnitState[] = [];
  let p = { x: from.x + dir.x, y: from.y + dir.y };
  while (inBounds(p, terrain)) {
    const occ = units.find((u) => u.hp > 0 && u.pos.x === p.x && u.pos.y === p.y);
    if (occ && occ.faction !== self.faction) out.push(occ);
    p = { x: p.x + dir.x, y: p.y + dir.y };
  }
  return out;
}
function rayCellsFrom(from: Vec2, dir: Vec2, terrain: TerrainGrid): Vec2[] {
  const cells: Vec2[] = [];
  let p = { x: from.x + dir.x, y: from.y + dir.y };
  while (inBounds(p, terrain)) {
    cells.push({ ...p });
    p = { x: p.x + dir.x, y: p.y + dir.y };
  }
  return cells;
}
function bestLinePick(
  self: UnitState,
  from: Vec2,
  units: UnitState[],
```

```
  /**
   * 显示冷却剩余回合。布阵页预览敌人时关掉：那时候还没开打，
   * 写「剩 0 回合」是在回答一个玩家没问的问题。
   */
  showCooldown: boolean;
}
/**
 * 战场单位（含布阵页的敌人预览）。
 *
 * 数值一律走 `effectiveUnitDef`--它是战斗结算读的同一个函数，
 * 所以面板上的攻击力就是这一刻真会打出去的攻击力，含限时增益减益。
 * 自己抄一遍加减法的话，加了新 buff 种类时面板会悄悄开始说谎。
 */
export function battleUnitInfoModel(u: UnitState, opts: BattleUnitInfoOptions): UnitInfoModel {
  const ed = effectiveUnitDef(u, UNIT_DEFS);
  const kindName = UNIT_DEFS[u.defId].name;
  const skills: UnitInfoSkillSection[] = [];
  const cdNote = (left: number): string | undefined =>
    opts.showCooldown && left > 0 ? `（剩 ${left}）` : undefined;
  if (ed.skill) {
    const s = mainSection(ed.skill.id, u.skillMods, cdNote(u.skillCd));
    if (s) {
      // 敌方技能皮肤覆写展示名/图标；结算仍认 ed.skill.id -> SkillSpec
      if (u.battleSkill?.name) s.name = u.battleSkill.name;
      if (u.battleSkill?.iconKey) s.iconKey = u.battleSkill.iconKey;
      skills.push(s);
    }
  }
  if (u.tempSkill) {
    const s = tempSection(u.tempSkill.id, cdNote(u.tempSkillCd ?? 0));
    if (s) skills.push(s);
  }
  const statuses = (u.timedBattleEffects ?? [])
    .filter((e) => e.roundsLeft > 0)
    .map(describeTimedEffect);
  // 敌方副标题写「定位」而不是兵种名：草原杂兵的 defId 还是 sword/bow，
  // 但外观和名字已经是魔物了，面板上冒出「剑士」等于把实现细节漏给玩家。
  // 我方相反，职业名正是玩家认人的方式。
  const role = ed.isRanged ? '远程' : '近战';
  const subtitle = u.faction === 'enemy'
    ? `${u.boss ? '首领' : '敌方'} · ${role}`
    : kindName;
  return {
    name: u.displayName ?? kindName,
    subtitle,
    createPortrait: () => createUnitToken(u.animSet ?? u.defId, u.faction, 48),
    stats: [
      { label: '生命', value: `${Math.max(0, u.hp)}/${ed.maxHp}` },
      { label: '攻击', value: `${ed.atk}` },
      { label: '速度', value: `${ed.spd}` },
```

```
    { label: '移动', value: `${ed.move}` },
   ],
   strikeTitle: '普通攻击',
   strike: [
    { label: '射程', value: `${ed.range}` },
    { label: '类型', value: ed.isRanged ? '远程' : '近战' },
    { label: '嘲讽', value: ed.taunt ? '是' : '否' },
   ],
   skills,
   statuses: statuses.length > 0 ? statuses : undefined,
  };
}
import * as PIXI from 'pixi.js';
import { makeText, textStyle } from '@/theme/typography';
import type { SkillSpec } from '@/data/skillCatalog';
import { describeReach, describeSkillSpec } from '@/data/skillText';
import { getSkillMod, isExclusiveMod, type SkillModRarity } from '@/data/skillModCatalog';
import { createUiIcon } from '@/view/renderHelpers';
/**
 * 单位信息面板：布阵页和战斗页共用同一块渲染。
 *
 * 两边看的是同一件事--「这个人现在什么状态、带什么招」。做成两份的话，
 * 迟早出现布阵页写着一个数、战斗里点开写着另一个数，而玩家没法判断该信谁。
 * 所以这里只认一个**展示模型**，数值怎么算是调用方（`unitInfoModel.ts`）的事。
 */
export interface UnitInfoStat {
 label: string;
 value: string;
}
export interface UnitInfoSkillSection {
 /** 段标题，如「装备技能」「临时技能（本局）」 */
 title: string;
 name: string;
 nameColor: number;
 iconKey: string;
 /** **折进词条之后**的规格：面板写的必须就是战斗里结算的 */
 spec: SkillSpec;
 /** 原始规格，只用来标出冷却被词条缩短了 */
 baseSpec: SkillSpec;
 /** 冷却行后面的补充，如战斗中的「剩 2 回合」 */
 cooldownNote?: string;
 /** 画范围格子图。临时技能不画，否则面板长到要滚动 */
 showRange: boolean;
 extraDesc?: string[];
 /** 「本局加成」清单挂在这一段的格子图旁边；只给主技能段 */
 modIds?: readonly string[];
}
export interface UnitInfoModel {
 name: string;
 subtitle: string;
```

```
  /**
   * 48px 头像的**工厂**，不是画好的对象。
   *
   * 模型要能在没有 DOM 的环境里构造（单测拿它核对面板数值和战斗结算是否一致），
   * 而 `createUnitToken` 会 new Graphics，那一步必须推迟到真要渲染的时候。
   */
  createPortrait: () => PIXI.Container;
  stats: UnitInfoStat[];
  strikeTitle: string;
  strike: UnitInfoStat[];
  skills: UnitInfoSkillSection[];
  /** 战斗中生效的限时状态，如「中毒: 每回合 -6 血（剩 2 回合）」 */
  statuses?: string[];
}
const LABEL_STYLE = textStyle('caption', { fill: 0xaaa088 });
const VALUE_STYLE = textStyle('uiStrong', { fill: 0x3a3a2a, fontSize: 12 });
const SECTION_STYLE = textStyle('uiStrong', { fill: 0x6b4c2a, fontSize: 13 });
const LINE_H = 20;
/** 词条稀有度 -> 米色面板上的字色（三选一卡是深底，那套色直接搬过来会发飘） */
const MOD_TEXT_COLOR: Record<SkillModRarity, number> = {
  common: 0x5a6a7a,
  rare: 0x2f6fae,
  epic: 0xa5561f,
};
/**
 * 「本局加成」清单：图标 + 名称×层数 + 这一层的实际效果。
 *
 * 只写名字不够--「锋锐×2」到底是 +25% 还是 +50%，玩家没法从名字里读出来，
 * 而这正是他决定要不要再叠一层的依据，所以每条都带上按层数算好的描述。
 *
 * 挂不上当前技能的词条（比如给纯治疗技挂「淬毒」）也列，压暗并注明原因：
 * 词条是跟着角色走的，换回能用的技能它就活了，直接藏掉会让玩家以为东西丢了。
 */
function buildRunModList(
  modIds: readonly string[],
  spec: SkillSpec,
  width: number,
): PIXI.Container | null {
  if (modIds.length === 0) return null;
  const counted = new Map<string, number>();
  for (const id of modIds) counted.set(id, (counted.get(id) ?? 0) + 1);
  const box = new PIXI.Container();
  const title = makeText('本局加成', 'caption', { fill: 0x6b4c2a, fontSize: 10, fontWeight: 'bold' });
  box.addChild(title);
  let y = title.height + 4;
  for (const [modId, rawN] of counted) {
    const mod = getSkillMod(modId);
    if (!mod) continue;
    const n = Math.min(rawN, mod.maxStacks);
    const live = mod.canApply(spec);
```

```
    const row = new PIXI.Container();
    row.y = y;
    row.alpha = live ? 1 : 0.5;
    const icon = createUiIcon(mod.icon, 14);
    if (icon) {
      icon.y = -1;
      row.addChild(icon);
    }
    const textX = icon ? 18 : 0;
    // 专属词条标出来：它换个技能就再也拿不到，玩家换招前得知道自己会丢什么
    const label = isExclusiveMod(mod) ? `${mod.name}（专属）` : n > 1 ? `${mod.name}×${n}` : mod.name;
    const name = makeText(label, 'caption', {
      fill: live ? MOD_TEXT_COLOR[mod.rarity] : 0x8a8a7a,
      fontSize: 10,
      fontWeight: 'bold',
    });
    name.x = textX;
    row.addChild(name);
    const desc = makeText(live ? mod.describe(n) : '当前技能用不到，换回可用技能即恢复', 'micro', {
      fill: 0x7a7a6a,
      lineHeight: 12,
      wordWrap: true,
      wordWrapWidth: Math.max(60, width - textX),
    });
    desc.x = textX;
    desc.y = name.height + 1;
    row.addChild(desc);
    box.addChild(row);
    y += name.height + 1 + desc.height + 5;
  }
  // 一条都挂不上时只剩个标题，不如不画
  return box.children.length > 1 ? box : (box.destroy({ children: true }), null);
}
type CellKind = 'empty' | 'center' | 'hit' | 'ray';
/** 技能范围示意格 + 右侧的本局加成 + 下方的范围说明与图例 */
function buildRangeRow(
  spec: SkillSpec,
  modIds: readonly string[] | undefined,
  panelW: number,
  onTick: (cells: PIXI.Graphics[]) => void,
): { view: PIXI.Container; height: number } {
  const shape = spec.shape;
  const cs = 12;
  const gap = 1;
  const st2 = cs + gap;
  let gridR = 2;
  let rangeDesc = '';
  const isLine = shape.type === 'lineBestRayAllFoes';
  if (shape.type === 'neighborAoE') {
    gridR = shape.manhattan + 1;
```

```
    rangeDesc = `周围${shape.manhattan}格范围\n命中所有敌人`;
  } else if (shape.type === 'discAoE') {
    // 「横扫」「势不可挡」把环摊成整片圆，格子图必须跟着变--
    // 词条改了形状却画老图的话，玩家会照着错的范围走位。
    gridR = shape.radius + 1;
    rangeDesc = `周围${shape.radius}格全覆盖\n命中所有敌人`;
  } else if (shape.type === 'neighborPickLowest') {
    gridR = shape.manhattan + 1;
    rangeDesc = `${describeReach(shape.manhattan, 'exact')}\n选中血量最低的敌人`;
  } else if (shape.type === 'neighborPickFoe') {
    gridR = shape.manhattan + 1;
    const pickLabel = shape.pick === 'lowestHp' ? '血量最低' : '血量最高';
    rangeDesc = `${describeReach(shape.manhattan, shape.reach)}\n选中${pickLabel}的敌人`;
  } else if (shape.type === 'neighborPickAlly') {
    gridR = shape.manhattan + 1;
    const pickLabel = shape.pick === 'lowestHp' ? '血量最低' : '血量最高';
    rangeDesc = `周围${shape.manhattan}格范围\n选中${pickLabel}的友方`;
  } else if (shape.type === 'selfCast') {
    gridR = 1;
    rangeDesc = '对自己释放\n无需选择目标';
  } else if (isLine) {
    gridR = 3;
    rangeDesc = '上下左右四方向\n射线穿透所有敌人';
  }
  const gridD = gridR * 2 + 1;
  const cells: CellKind[][] = [];
  for (let gy = 0; gy < gridD; gy++) {
    cells.push([]);
    for (let gx = 0; gx < gridD; gx++) cells[gy]!.push('empty');
  }
  cells[gridR]![gridR] = 'center';
  if (shape.type === 'neighborAoE' || shape.type === 'neighborPickLowest'
      || shape.type === 'neighborPickFoe' || shape.type === 'neighborPickAlly'
      || shape.type === 'discAoE') {
    const md = shape.type === 'discAoE' ? shape.radius : shape.manhattan;
    /*
     * 整片（曼哈顿 <= md）还是一圈环（正好 = md）。
     *
     * 这里原先无条件按整片画，而除 `discAoE` 和 `reach: 'within'` 以外的形状都是环：
     * 「长驱突刺」（正好 2 格）的图因此多画了 4 个贴脸格，玩家照图走到敌人旁边，
     * 技能却点不出来。md 为 1 时环和整片恰好一样，所以这个错一直被邻格技能盖着，
     * 只有 2 格以上的技能会露出来。
     */
    const solid = shape.type === 'discAoE'
      || (shape.type === 'neighborPickFoe' && shape.reach === 'within');
    for (let dy = -md; dy <= md; dy++) {
      for (let dx = -md; dx <= md; dx++) {
        if (dx === 0 && dy === 0) continue;
        const d = Math.abs(dx) + Math.abs(dy);
        if (solid ? d <= md : d === md) cells[gridR + dy]![gridR + dx] = 'hit';
```

```
    }
   }
  } else if (isLine) {
   for (const [ddx, ddy] of [[0, -1], [0, 1], [-1, 0], [1, 0]]) {
    for (let s = 1; s <= gridR; s++) {
     const gx = gridR + ddx! * s;
     const gy = gridR + ddy! * s;
     if (gx >= 0 && gx < gridD && gy >= 0 && gy < gridD) cells[gy]![gx] = 'ray';
    }
   }
  }
  const gridTotalW = gridD * st2 - gap;
  const row = new PIXI.Container();
  const gridContainer = new PIXI.Container();
  const hitCells: PIXI.Graphics[] = [];
  for (let gy = 0; gy < gridD; gy++) {
   for (let gx = 0; gx < gridD; gx++) {
    const kind = cells[gy]![gx]!;
    const px = gx * st2;
    const py = gy * st2;
    const cell = new PIXI.Graphics();
    if (kind === 'center') {
     cell.beginFill(0x4488cc, 0.85);
     cell.drawRoundedRect(px, py, cs, cs, 2);
     cell.endFill();
    } else if (kind === 'hit' || kind === 'ray') {
     cell.beginFill(kind === 'ray' ? 0xdd6633 : 0xcc3333, 0.7);
     cell.drawRoundedRect(px, py, cs, cs, 2);
     cell.endFill();
     hitCells.push(cell);
    } else {
     cell.lineStyle(1, 0xccccbb, 0.25);
     cell.beginFill(0xeeeedd, 0.12);
     cell.drawRoundedRect(px, py, cs, cs, 1);
     cell.endFill();
    }
    gridContainer.addChild(cell);
   }
  }
  // 射线技能在四个边缘画箭头，表示延伸
  if (isLine) {
   const arrowStyle = textStyle('micro', { fill: 0xdd6633, fontSize: 8 });
   const arrowOffsets: [number, number, string][] = [
    [gridR * st2 + cs / 2, -6, '▲'],
    [gridR * st2 + cs / 2, gridD * st2 - gap + 1, 'v'],
    [-6, gridR * st2 + cs / 2, '<'],
    [gridD * st2 - gap + 2, gridR * st2 + cs / 2, '>'],
   ];
   for (const [ax, ay, ch] of arrowOffsets) {
    const ar = new PIXI.Text(ch, arrowStyle);
```

```
    ar.anchor.set(0.5);
    ar.x = ax;
    ar.y = ay;
    gridContainer.addChild(ar);
  }
 }
 gridContainer.x = 16;
 row.addChild(gridContainer);
 const colX = 16 + gridTotalW + 12;
 const colW = panelW - 12 - colX;
 // 本局词条：占掉格子图右边那块空地。
 //
 // 之前只有背包里能翻到，而玩家真正想核对「这一局攒了什么」的时机，恰恰是开打前
 // 对着这个面板看阵容。摆在技能格子旁边还顺带说清了归属：改的就是上面那一招。
 const modsBlock = modIds ? buildRunModList(modIds, spec, colW) : null;
 // 词条把右列占了的话，范围说明和图例挪到格子下面排成一行。
 // 反过来把词条压到下面是不行的：右列只剩说明文字下面那几像素，
 // 一条「技能伤害提升 50%」就要折三行。
 const descBelow = modsBlock !== null;
 const rangeDescTx = makeText(descBelow ? rangeDesc.replace(/\n/g, ' · ') : rangeDesc, 'caption', {
  fill: 0x6a6a5a, fontSize: 10, lineHeight: 15,
  wordWrap: true, wordWrapWidth: descBelow ? panelW - 32 : colW,
 });
 if (descBelow) {
  rangeDescTx.x = 16;
  // 让到词条那一列的下面，而不是只让过格子图：词条攒多了会比格子高，
  // 按格子高度排会直接压在最后一条词条上。
  rangeDescTx.y = Math.max(gridTotalW, modsBlock?.height ?? 0) + 8;
 } else {
  rangeDescTx.x = colX;
  rangeDescTx.y = Math.max(0, (gridTotalW - rangeDescTx.height) / 2);
 }
 row.addChild(rangeDescTx);
 if (modsBlock) {
  modsBlock.x = colX;
  row.addChild(modsBlock);
 }
 // 图例。跟着说明走：在右列时排在它下面，在下方时右对齐到同一行
 const legendY = descBelow
  ? rangeDescTx.y + Math.max(0, rangeDescTx.height - 12)
  : rangeDescTx.y + rangeDescTx.height + 6;
 const legendX = descBelow
  ? Math.max(rangeDescTx.x + rangeDescTx.width + 12, panelW - 12 - 78)
  : colX;
 const legCenterDot = new PIXI.Graphics();
 legCenterDot.beginFill(0x4488cc, 0.85);
 legCenterDot.drawRoundedRect(0, 0, 8, 8, 2);
 legCenterDot.endFill();
 legCenterDot.x = legendX;
 legCenterDot.y = legendY;
```

```
  row.addChild(legCenterDot);
  const legCenterTx = makeText('自身', 'micro', { fill: 0x888877 });
  legCenterTx.x = legendX + 12;
  legCenterTx.y = legendY - 1;
  row.addChild(legCenterTx);
  const legHitDot = new PIXI.Graphics();
  legHitDot.beginFill(isLine ? 0xdd6633 : 0xcc3333, 0.7);
  legHitDot.drawRoundedRect(0, 0, 8, 8, 2);
  legHitDot.endFill();
  legHitDot.x = legCenterTx.x + legCenterTx.width + 10;
  legHitDot.y = legendY;
  row.addChild(legHitDot);
  const legHitTx = makeText('范围', 'micro', { fill: 0x888877 });
  legHitTx.x = legHitDot.x + 12;
  legHitTx.y = legendY - 1;
  row.addChild(legHitTx);
  onTick(hitCells);
  const height = Math.max(
    gridTotalW,
    legendY + 14,
    rangeDescTx.y + rangeDescTx.height,
    modsBlock ? modsBlock.height : 0,
  );
  return { view: row, height };
}
function separator(panelW: number, y: number): PIXI.Graphics {
  const g = new PIXI.Graphics();
  g.lineStyle(1, 0xd0c8b8, 0.6);
  g.moveTo(12, y);
  g.lineTo(panelW - 12, y);
  return g;
}
/** 两列键值区，返回占用高度 */
function addStatGrid(
  panel: PIXI.Container,
  items: UnitInfoStat[],
  panelW: number,
  top: number,
): number {
  const colW = Math.floor((panelW - 24) / 2);
  for (let i = 0; i < items.length; i++) {
    const s = items[i]!;
    const sx = 16 + (i % 2) * colW;
    const sy = top + Math.floor(i / 2) * LINE_H;
    const lb = new PIXI.Text(s.label, LABEL_STYLE);
    lb.x = sx;
    lb.y = sy;
    panel.addChild(lb);
    const vl = new PIXI.Text(s.value, VALUE_STYLE);
    vl.x = sx + 36;
```

```
    vl.y = sy;
    panel.addChild(vl);
  }
  return Math.ceil(items.length / 2) * LINE_H;
}
export interface UnitInfoPanel {
  view: PIXI.Container;
  height: number;
  /** 范围格子的呼吸动画；面板销毁时要调 */
  stop(): void;
}
export interface UnitInfoPanelOptions {
  /**
   * 画自己的米白底（默认画）。
   *
   * 嵌进另一个米白面板时关掉：两层同色圆角矩形叠在一起只会在拐角处露出一圈
   * 深浅不一的边，读起来像没对齐。角色页的详情弹窗就是这种嵌套。
   */
  drawBg?: boolean;
}
export function createUnitInfoPanel(
  model: UnitInfoModel,
  panelW: number,
  opts?: UnitInfoPanelOptions,
): UnitInfoPanel {
  const panel = new PIXI.Container();
  panel.eventMode = 'static';
  const tickers: (() => void)[] = [];
  let cy = 16;
  const portrait = model.createPortrait();
  portrait.x = 30;
  portrait.y = cy + 24;
  panel.addChild(portrait);
  const nameTx = makeText(model.name, 'title', { fill: 0x3a3a2a, fontSize: 16 });
  nameTx.x = 62;
  nameTx.y = cy + 6;
  panel.addChild(nameTx);
  const subTx = makeText(model.subtitle, 'body', { fill: 0x8a7a5a });
  subTx.x = 62;
  subTx.y = cy + 28;
  panel.addChild(subTx);
  cy += 56;
  panel.addChild(separator(panelW, cy));
  cy += 8;
  const secBase = new PIXI.Text('基础属性', SECTION_STYLE);
  secBase.x = 12;
  secBase.y = cy;
  panel.addChild(secBase);
  cy += LINE_H + 2;
  cy += addStatGrid(panel, model.stats, panelW, cy) + 8;
```

```
  // 限时状态。战斗中点开才有内容--「他为什么突然打这么疼」只能在这里回答。
  if (model.statuses?.length) {
    panel.addChild(separator(panelW, cy));
    cy += 8;
    const secSt = new PIXI.Text('当前状态', SECTION_STYLE);
    secSt.x = 12;
    secSt.y = cy;
    panel.addChild(secSt);
    cy += LINE_H + 2;
    const tx = makeText(model.statuses.join('\n'), 'caption', {
      fill: 0x555544, fontSize: 10, lineHeight: 16,
      wordWrap: true, wordWrapWidth: panelW - 32,
    });
    tx.x = 16;
    tx.y = cy;
    panel.addChild(tx);
    cy += tx.height + 8;
  }
  panel.addChild(separator(panelW, cy));
  cy += 8;
  const secStrike = new PIXI.Text(model.strikeTitle, SECTION_STYLE);
  secStrike.x = 12;
  secStrike.y = cy;
  panel.addChild(secStrike);
  cy += LINE_H + 2;
  cy += addStatGrid(panel, model.strike, panelW, cy) + 8;
  for (const sk of model.skills) {
    panel.addChild(separator(panelW, cy));
    cy += 8;
    const sec = new PIXI.Text(sk.title, SECTION_STYLE);
    sec.x = 12;
    sec.y = cy;
    panel.addChild(sec);
    cy += LINE_H + 2;
    // 技能图标：战斗操作条、三选一卡片、这里用的是同一张图。
    // 玩家在这三处认的是同一个符号，换到战斗里那排无字圆钮才不用重新学。
    const icon = createUiIcon(sk.iconKey, 22);
    if (icon) {
      icon.x = 16;
      icon.y = cy - 3;
      panel.addChild(icon);
    }
    const nameX = icon ? 42 : 16;
    const skName = makeText(sk.name, 'uiStrong', { fill: sk.nameColor, fontSize: 13 });
    skName.x = nameX;
    skName.y = cy;
    panel.addChild(skName);
    const cdText = `CD: ${sk.spec.cooldown}回合${sk.cooldownNote ?? ''}`;
    const cdTx = makeText(cdText, 'caption', {
      fill: sk.spec.cooldown < sk.baseSpec.cooldown ? 0x3a8a5a : 0x888888,
```

```
  });
  cdTx.x = nameX + skName.width + 10;
  cdTx.y = cy + 2;
  panel.addChild(cdTx);
  cy += LINE_H;
  const descTx = makeText([...describeSkillSpec(sk.spec), ...(sk.extraDesc ?? [])].join('\n'), ...
    fill: 0x555544, fontSize: 10, lineHeight: 16,
    wordWrap: true, wordWrapWidth: panelW - 32,
  });
  descTx.x = 16;
  descTx.y = cy;
  panel.addChild(descTx);
  cy += descTx.height + 8;
  if (sk.showRange) {
    const { view, height } = buildRangeRow(sk.spec, sk.modIds, panelW, (hitCells) => {
      let phase = 0;
      tickers.push(() => {
        phase += 0.06;
        const a = 0.45 + 0.35 * Math.sin(phase);
        for (const c of hitCells) c.alpha = a;
      });
    });
    view.y = cy;
    panel.addChild(view);
    cy += height + 8;
  }
}
cy += 12;
if (opts?.drawBg ?? true) {
  const bg = new PIXI.Graphics();
  bg.beginFill(0xfefef6, 0.97);
  bg.drawRoundedRect(0, 0, panelW, cy, 14);
  bg.endFill();
  panel.addChildAt(bg, 0);
}
// 场景可能被 SceneManager 整棵拆掉而不经过 `stop()`（战斗结束时面板还开着），
// 所以 ticker 自己也要能发现容器没了并摘掉自己，否则它会一直抓着这棵树不放。
const tick = (): void => {
  if (panel.destroyed) {
    PIXI.Ticker.shared.remove(tick);
    return;
  }
  for (const t of tickers) t();
};
if (tickers.length > 0) PIXI.Ticker.shared.add(tick);
return {
  view: panel,
  height: cy,
  stop: () => PIXI.Ticker.shared.remove(tick),
};
```

```
}
/**
 * 遮罩 + 居中面板 + 点遮罩关闭。两个页面都要这一套，别各写一遍。
 *
 * 面板自己吃掉点击（`eventMode: 'static'`），否则点面板正文也会把它关掉。
 */
export function createUnitInfoOverlay(
  model: UnitInfoModel,
  screenW: number,
  screenH: number,
  onClose: () => void,
): { view: PIXI.Container; stop(): void } {
  const root = new PIXI.Container();
  const dim = new PIXI.Graphics();
  dim.beginFill(0x000000, 0.5);
  dim.drawRect(0, 0, screenW, screenH);
  dim.endFill();
  dim.eventMode = 'static';
  dim.on('pointertap', onClose);
  root.addChild(dim);
  const panelW = Math.min(300, screenW - 32);
  const panel = createUnitInfoPanel(model, panelW);
  panel.view.x = Math.floor((screenW - panelW) / 2);
  // 面板可能比屏幕高（词条攒满 + 长技能说明），那就贴顶而不是居中溢出到看不见
  panel.view.y = Math.max(8, Math.floor((screenH - panel.height) / 2));
  root.addChild(panel.view);
  return { view: root, stop: panel.stop };
}
import * as PIXI from 'pixi.js';
import { makeText } from '@/theme/typography';
export interface UnitOverheadOptions {
  maxHp: number;
  currentHp: number;
  professionName: string;
  faction: 'player' | 'enemy';
  cell: number;
}
export interface UnitOverheadHandle {
  readonly root: PIXI.Container;
  updateHp: (currentHp: number, maxHp?: number) => void;
  destroy: () => void;
}
const HP_BG = 0x1a0505;
const PLAYER_HP_BORDER = 0x2d8a2d;
const PLAYER_HP_FILL = 0x44bb44;
const ENEMY_HP_BORDER = 0xcc3333;
const ENEMY_HP_FILL = 0xb82a2a;
/**
 * 单位头顶：血条 + 职业名（同一行，职业在血条右侧）。
 * 整体在角色上方，本地坐标 y=0 为底边。
```

```
 */
export function createUnitOverhead(opts: UnitOverheadOptions): UnitOverheadHandle {
  const root = new PIXI.Container();
  let maxHp = Math.max(1, opts.maxHp);
  let curHp = Math.min(Math.max(0, opts.currentHp), maxHp);
  const isPlayer = opts.faction === 'player';
  const hpBorder = isPlayer ? PLAYER_HP_BORDER : ENEMY_HP_BORDER;
  const hpFillColor = isPlayer ? PLAYER_HP_FILL : ENEMY_HP_FILL;
  const barW = Math.max(18, Math.floor(opts.cell * 0.52));
  const barH = Math.max(3, Math.floor(opts.cell * 0.09));
  const labelFs = Math.max(7, Math.min(10, Math.floor(opts.cell * 0.18)));
  const label = makeText(opts.professionName, 'combatLabel', {
    fill: 0xffffff,
    fontSize: labelFs,
  });
  label.anchor.set(0, 0.5);
  const totalW = barW + 3 + label.width;
  const hpBg = new PIXI.Graphics();
  const hpFill = new PIXI.Graphics();
  function redrawBar(): void {
    hpBg.clear();
    hpFill.clear();
    const x0 = -totalW / 2;
    const y0 = -barH;
    hpBg.lineStyle(1, hpBorder, 1);
    hpBg.beginFill(HP_BG, 0.95);
    hpBg.drawRoundedRect(x0, y0, barW, barH, 2);
    hpBg.endFill();
    const ratio = maxHp > 0 ? Math.max(0, Math.min(1, curHp / maxHp)) : 0;
    const innerW = barW - 4;
    const fillW = Math.max(0, innerW * ratio);
    hpFill.beginFill(hpFillColor, 1);
    hpFill.drawRoundedRect(x0 + 2, y0 + 1, fillW, barH - 2, 1);
    hpFill.endFill();
  }
  label.x = -totalW / 2 + barW + 3;
  label.y = -barH / 2;
  root.addChild(hpBg);
  root.addChild(hpFill);
  root.addChild(label);
  redrawBar();
  return {
    root,
    updateHp(currentHp: number, nextMax?: number): void {
      if (nextMax !== undefined) maxHp = Math.max(1, nextMax);
      curHp = Math.min(Math.max(0, currentHp), maxHp);
      redrawBar();
    },
    destroy(): void {
      root.destroy({ children: true });
```

```
    },
  };
}
/**
 * Image 构造函数模拟
 */
const platform = require('./platform');
function Image() {
  return platform.createImage();
}
module.exports = Image;
/**
 * 触摸事件适配
 * 将小游戏触摸事件转换为 PixiJS 所需的标准 DOM 事件格式
 */
const platform = require('./platform');
const { canvas } = require('./canvas');
class TouchEvent {
  constructor(type, touches) {
    this.type = type;
    this.target = canvas;
    this.currentTarget = canvas;
    this.touches = touches || [];
    this.changedTouches = touches || [];
    this.targetTouches = touches || [];
    this.timeStamp = Date.now();
    this.bubbles = true;
    this.cancelable = true;
    this.defaultPrevented = false;
  }
  preventDefault() {
    this.defaultPrevented = true;
  }
  stopPropagation() {}
}
// 将小游戏触摸坐标转换为 canvas 坐标
function convertTouches(rawTouches) {
  if (!rawTouches) return [];
  return Array.prototype.map.call(rawTouches, (touch) => ({
    identifier: touch.identifier,
    clientX: touch.clientX,
    clientY: touch.clientY,
    pageX: touch.clientX,
    pageY: touch.clientY,
    screenX: touch.clientX,
    screenY: touch.clientY,
    target: canvas,
  }));
}
// 注册触摸事件监听桥接
```

```
function registerTouchEvents() {
  const _listeners = {};
  // 给 canvas 挂上 addEventListener / removeEventListener
  canvas.addEventListener = function(type, handler, options) {
    if (!_listeners[type]) _listeners[type] = [];
    _listeners[type].push(handler);
  };
  canvas.removeEventListener = function(type, handler) {
    if (!_listeners[type]) return;
    const idx = _listeners[type].indexOf(handler);
    if (idx !== -1) _listeners[type].splice(idx, 1);
  };
  // 分发事件到 canvas 上的监听器
  function dispatch(type, rawEvent) {
    const touches = convertTouches(rawEvent.touches || rawEvent.changedTouches);
    const event = new TouchEvent(type, touches);
    event.changedTouches = convertTouches(rawEvent.changedTouches);
    const queue = _listeners[type];
    if (queue) {
      queue.forEach(handler => {
        try { handler(event); } catch (e) { console.error('[TouchEvent]', type, e); }
      });
    }
  }
  // pointer 事件映射（PixiJS 7 优先使用 pointer 事件）
  function dispatchPointer(pointerType, rawEvent) {
    const touches = rawEvent.changedTouches || rawEvent.touches || [];
    if (!touches.length) return;
    const touch = touches[0];
    const pointerEvent = {
      type: pointerType,
      pointerId: touch.identifier || 0,
      pointerType: 'touch',
      clientX: touch.clientX,
      clientY: touch.clientY,
      pageX: touch.clientX,
      pageY: touch.clientY,
      screenX: touch.clientX,
      screenY: touch.clientY,
      x: touch.clientX,
      y: touch.clientY,
      offsetX: touch.clientX,
      offsetY: touch.clientY,
      width: 1,
      height: 1,
      pressure: pointerType === 'pointerup' ? 0 : 0.5,
      button: 0,
      buttons: pointerType === 'pointerup' ? 0 : 1,
      isPrimary: true,
      target: canvas,
```

```
    currentTarget: canvas,
    timeStamp: Date.now(),
    bubbles: true,
    cancelable: true,
    preventDefault() {},
    stopPropagation() {},
    stopImmediatePropagation() {},
   };
   const queue = _listeners[pointerType];
   if (queue) {
    queue.forEach(handler => {
     try { handler(pointerEvent); } catch (e) { console.error('[PointerEvent]', pointerType, e...
    });
   }
  }
  // 分发 window 上的全局事件（PixiJS EventSystem 在 window 上也注册了 pointermove/pointerup）
  function dispatchToWindow(type, event) {
   if (typeof GameGlobal !== 'undefined' && GameGlobal.__windowDispatchEvent) {
    GameGlobal.__windowDispatchEvent(type, event);
   }
  }
  // 诊断：3秒后输出已注册的 canvas 事件监听器类型
  setTimeout(function() {
   console.log('[TouchEvent] canvas listeners:', Object.keys(_listeners).join(', ') || '(空)');
  }, 3000);
  var _touchLogCount = 0;
  platform.onTouchStart((e) => {
   _touchLogCount++;
   if (_touchLogCount <= 5) {
    var t = (e.changedTouches || e.touches || [])[0];
    console.log('[Touch] down #' + _touchLogCount,
     'x:', t && t.clientX, 'y:', t && t.clientY,
     'canvasListeners:', Object.keys(_listeners).join(','));
   }
   dispatch('touchstart', e);
   dispatchPointer('pointerdown', e);
   // window 上也需要收到 pointerdown
   var touches = e.changedTouches || e.touches || [];
   if (touches.length) {
    var t = touches[0];
    dispatchToWindow('pointerdown', {
     type: 'pointerdown', pointerId: t.identifier || 0, pointerType: 'touch',
     clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
     button: 0, buttons: 1, isPrimary: true, target: canvas,
     preventDefault: function(){}, stopPropagation: function(){}, stopImmediatePropagation: fu...
    });
   }
  });
  var _moveLogCount = 0;
  platform.onTouchMove((e) => {
```

```
19358   dispatch('touchmove', e);
19359   dispatchPointer('pointermove', e);
19360   var touches = e.changedTouches || e.touches || [];
19361   if (touches.length) {
19362     var t = touches[0];
19363     _moveLogCount++;
19364     if (_moveLogCount <= 3) {
19365       console.log('[Touch] move #' + _moveLogCount,
19366         'x:', t.clientX, 'y:', t.clientY,
19367         'windowListeners(pointermove):', typeof GameGlobal.__windowDispatchEvent);
19368     }
19369     dispatchToWindow('pointermove', {
19370       type: 'pointermove', pointerId: t.identifier || 0, pointerType: 'touch',
19371       clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
19372       button: 0, buttons: 1, isPrimary: true, target: canvas,
19373       preventDefault: function(){}, stopPropagation: function(){}, stopImmediatePropagation: fu...
19374     });
19375   }
19376 });
19377 platform.onTouchEnd((e) => {
19378   dispatch('touchend', e);
19379   dispatchPointer('pointerup', e);
19380   var touches = e.changedTouches || [];
19381   if (touches.length) {
19382     var t = touches[0];
19383     dispatchToWindow('pointerup', {
19384       type: 'pointerup', pointerId: t.identifier || 0, pointerType: 'touch',
19385       clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
19386       button: 0, buttons: 0, isPrimary: true, target: canvas,
19387       preventDefault: function(){}, stopPropagation: function(){}, stopImmediatePropagation: fu...
19388     });
19389   }
19390 });
19391 platform.onTouchCancel((e) => {
19392   dispatch('touchcancel', e);
19393   dispatchPointer('pointercancel', e);
19394   var touches = e.changedTouches || [];
19395   if (touches.length) {
19396     var t = touches[0];
19397     dispatchToWindow('pointercancel', {
19398       type: 'pointercancel', pointerId: t.identifier || 0, pointerType: 'touch',
19399       clientX: t.clientX, clientY: t.clientY, pageX: t.clientX, pageY: t.clientY,
19400       button: 0, buttons: 0, isPrimary: true, target: canvas,
19401       preventDefault: function(){}, stopPropagation: function(){}, stopImmediatePropagation: fu...
19402     });
19403   }
19404 });
19405 // canvas.getBoundingClientRect - PixiJS 用来计算事件坐标
19406 // 必须返回逻辑像素尺寸（与触摸事件 clientX/clientY 一致），否则坐标偏移
19407 var sysInfo = platform.getSystemInfoSync();
```

```
var screenW = sysInfo.screenWidth || sysInfo.windowWidth || 375;
var screenH = sysInfo.screenHeight || sysInfo.windowHeight || 667;
try {
  canvas.getBoundingClientRect = function() {
    return {
      x: 0,
      y: 0,
      top: 0,
      left: 0,
      width: screenW,
      height: screenH,
      right: screenW,
      bottom: screenH,
    };
  };
} catch (e) {}
// clientWidth/clientHeight 也需返回逻辑像素
try {
  Object.defineProperty(canvas, 'clientWidth', { get: function() { return screenW; }, configura...
  Object.defineProperty(canvas, 'clientHeight', { get: function() { return screenH; }, configur...
} catch (e) {}
// PixiJS 会检查 canvas.style（微信 canvas 部分属性可能只读）
try {
  if (!canvas.style) canvas.style = {};
  canvas.style.touchAction = '';
  canvas.style.msTouchAction = '';
  canvas.style.cursor = '';
  canvas.style.width = screenW + 'px';
  canvas.style.height = screenH + 'px';
} catch (e) {}
// PixiJS 检查 focus
if (!canvas.focus) canvas.focus = function() {};
// PixiJS EventSystem 需要 canvas.parentElement 来 addEventListener
// 不能设为 null，设为一个带事件能力的虚拟节点
var _parentListeners = {};
var fakeParent = {
  addEventListener: function(type, handler, options) {
    if (!_parentListeners[type]) _parentListeners[type] = [];
    _parentListeners[type].push(handler);
  },
  removeEventListener: function(type, handler) {
    if (!_parentListeners[type]) return;
    var idx = _parentListeners[type].indexOf(handler);
    if (idx !== -1) _parentListeners[type].splice(idx, 1);
  },
};
try { canvas.parentElement = fakeParent; } catch (e) {
  try { Object.defineProperty(canvas, 'parentElement', { value: fakeParent, configurable: true,...
}
try { canvas.parentNode = fakeParent; } catch (e) {
```

```
    try { Object.defineProperty(canvas, 'parentNode', { value: fakeParent, configurable: true, wr...
  }
  // 诊断：确认 parentElement 是否设置成功
  console.log('[TouchEvent] canvas.parentElement set:', !!canvas.parentElement,
    ', getBoundingClientRect:', typeof canvas.getBoundingClientRect);
}
module.exports = { TouchEvent, registerTouchEvents };
/**
 * XMLHttpRequest 模拟
 * PixiJS 资源加载可能会用到
 */
const platform = require('./platform');
class XMLHttpRequest {
  constructor() {
    this.readyState = 0;
    this.status = 0;
    this.statusText = '';
    this.responseText = '';
    this.response = null;
    this.responseType = '';
    this.responseURL = '';
    this.withCredentials = false;
    this.timeout = 0;
    this._method = '';
    this._url = '';
    this._headers = {};
    this.onreadystatechange = null;
    this.onload = null;
    this.onerror = null;
    this.onabort = null;
    this.ontimeout = null;
    this.onprogress = null;
  }
  open(method, url) {
    this._method = method;
    this._url = url;
    this.readyState = 1;
  }
  setRequestHeader(key, value) {
    this._headers[key] = value;
  }
  getResponseHeader(key) {
    return this._responseHeaders ? this._responseHeaders[key.toLowerCase()] : null;
  }
  getAllResponseHeaders() {
    return '';
  }
  send(data) {
    const self = this;
    const responseType = this.responseType || 'text';
```

```
    platform.request({
      url: this._url,
      method: this._method || 'GET',
      header: this._headers,
      data: data || undefined,
      responseType: responseType === 'arraybuffer' ? 'arraybuffer' : 'text',
      dataType: responseType === 'json' ? 'json' : undefined,
      success(res) {
        self.status = res.statusCode;
        self.statusText = res.statusCode + '';
        self._responseHeaders = res.header || {};
        if (responseType === 'arraybuffer') {
          self.response = res.data;
        } else if (responseType === 'json') {
          self.response = res.data;
          self.responseText = JSON.stringify(res.data);
        } else {
          self.responseText = res.data;
          self.response = res.data;
        }
        self.readyState = 4;
        if (self.onreadystatechange) self.onreadystatechange();
        if (self.onload) self.onload();
      },
      fail(err) {
        self.status = 0;
        self.readyState = 4;
        if (self.onreadystatechange) self.onreadystatechange();
        if (self.onerror) self.onerror(err);
      },
    });
  }
  abort() {
    if (this.onabort) this.onabort();
  }
  addEventListener(type, handler) {
    this['on' + type] = handler;
  }
  removeEventListener() {}
}
// 静态常量
XMLHttpRequest.UNSENT = 0;
XMLHttpRequest.OPENED = 1;
XMLHttpRequest.HEADERS_RECEIVED = 2;
XMLHttpRequest.LOADING = 3;
XMLHttpRequest.DONE = 4;
module.exports = XMLHttpRequest;
/**
 * Canvas 管理模块
 * 第一次 createCanvas() 返回主屏 canvas（小游戏环境特性，微信和抖音一致）
```

```
 */
const platform = require('./platform');
const canvas = platform.createCanvas();
module.exports = { canvas };
/**
 * document 对象模拟
 */
const platform = require('./platform');
const Image = require('./Image');
const { Element } = require('./element');
const _eventListeners = {};
const body = new Element();
body.clientWidth = 0;
body.clientHeight = 0;
const documentElement = new Element();
const document = {
  body,
  documentElement,
  readyState: 'complete',
  createElement(tag) {
    tag = tag.toLowerCase();
    switch (tag) {
      case 'canvas':
        return platform.createCanvas();
      case 'img':
      case 'image':
        return platform.createImage();
      default:
        return new Element();
    }
  },
  createElementNS(_ns, tag) {
    return this.createElement(tag);
  },
  createTextNode() {
    return new Element();
  },
  getElementById() {
    return null;
  },
  getElementsByTagName(tag) {
    if (tag === 'canvas') {
      const { canvas } = require('./canvas');
      return [canvas];
    }
    return [];
  },
  querySelector() {
    return null;
  },
```

```
  querySelectorAll() {
    return [];
  },
  // PixiJS EventSystem 使用 elementFromPoint 来确定事件目标
  elementFromPoint() {
    const { canvas } = require('./canvas');
    return canvas;
  },
  addEventListener(type, handler) {
    if (!_eventListeners[type]) _eventListeners[type] = [];
    _eventListeners[type].push(handler);
  },
  removeEventListener(type, handler) {
    if (!_eventListeners[type]) return;
    const idx = _eventListeners[type].indexOf(handler);
    if (idx !== -1) _eventListeners[type].splice(idx, 1);
  },
  dispatchEvent(ev) {
    const queue = _eventListeners[ev.type];
    if (queue) queue.forEach(handler => handler(ev));
  },
  // PixiJS 会检查这些
  fonts: {
    add() {},
    delete() {},
    has() { return false; },
    forEach() {},
  },
  hidden: false,
  visibilityState: 'visible',
};
module.exports = document;
/**
 * DOM 元素模拟
 * 业界已知坑：HTMLCanvasElement / HTMLImageElement 必须用 constructor 直接赋值
 * 不能用 class extends，否则 instanceof 校验会失败
 */
const platform = require('./platform');
class Element {
  constructor() {
    this.childNodes = [];
    this.style = { cursor: null };
    this.clientWidth = 0;
    this.clientHeight = 0;
    this._listeners = {};
  }
  appendChild(child) {
    this.childNodes.push(child);
    return child;
  }
```

```
  removeChild(child) {
    const idx = this.childNodes.indexOf(child);
    if (idx !== -1) this.childNodes.splice(idx, 1);
    return child;
  }
  addEventListener(type, handler) {
    if (!this._listeners[type]) this._listeners[type] = [];
    this._listeners[type].push(handler);
  }
  removeEventListener(type, handler) {
    if (!this._listeners[type]) return;
    const idx = this._listeners[type].indexOf(handler);
    if (idx !== -1) this._listeners[type].splice(idx, 1);
  }
  insertBefore() {}
  replaceChild() {}
  cloneNode() { return new Element(); }
  setAttribute() {}
  getAttribute() { return null; }
  getBoundingClientRect() {
    return { x: 0, y: 0, top: 0, left: 0, width: 0, height: 0, right: 0, bottom: 0 };
  }
}
// 通过 constructor 直接赋值（非 extends），确保 instanceof 正确
const HTMLCanvasElement = platform.createCanvas().constructor;
const HTMLImageElement = platform.createImage().constructor;
class HTMLVideoElement extends Element {}
module.exports = {
  Element,
  HTMLCanvasElement,
  HTMLImageElement,
  HTMLVideoElement,
};
/**
 * pixi-adapter 统一入口
 * 根据运行环境（真机/模拟器）将 DOM 模拟对象挂载到全局
 *
 * 关键：真机环境中 IIFE bundle 的自由变量（document、window 等）
 * 必须在 JS 引擎的全局作用域中可达，仅挂到 GameGlobal 不够--
 * GameGlobal 只是跨文件共享对象，不是全局作用域。
 * 因此真机需要挂到 globalThis / global 上。
 */
const platform = require('./platform');
const { noop } = require('./util');
const Image = require('./Image');
const { canvas } = require('./canvas');
const location = require('./location');
const document = require('./document');
const navigator = require('./navigator');
const localStorage = require('./localStorage');
```

```
const XMLHttpRequest = require('./XMLHttpRequest');
const { registerTouchEvents } = require('./TouchEvent');
const {
  Element,
  HTMLCanvasElement,
  HTMLImageElement,
  HTMLVideoElement,
} = require('./element');
// ======== 获取真正的 JS 全局对象 ========
// 优先 globalThis（ES2020+），其次 global（Node/V8），最后 GameGlobal
const _realGlobal = (typeof globalThis !== 'undefined' && globalThis)
  || (typeof global !== 'undefined' && global)
  || GameGlobal;
// ======== Patch Object.defineProperty ========
const _origDefineProperty = Object.defineProperty;
Object.defineProperty = function safeDefineProperty(obj, prop, descriptor) {
  try {
    return _origDefineProperty.call(Object, obj, prop, descriptor);
  } catch (e) {
    if (e instanceof TypeError) return obj;
    throw e;
  }
};
const _origDefineProperties = Object.defineProperties;
Object.defineProperties = function safeDefineProperties(obj, props) {
  for (const key in props) {
    if (Object.prototype.hasOwnProperty.call(props, key)) {
      try {
        _origDefineProperty.call(Object, obj, key, props[key]);
      } catch (e) {
        if (!(e instanceof TypeError)) throw e;
      }
    }
  }
  return obj;
};
// ======== 获取系统信息 ========
const sysInfo = platform.getSystemInfoSync();
const isDevtools = sysInfo.platform === 'devtools';
// ======== 定时器 & 动画帧 polyfill ========
// 真机 IIFE bundle 可能无法以自由变量访问这些 API，
// 在 adapter 模块作用域中它们可用，挂到真正的全局对象。
;(function _patchTimers() {
  var pairs = {};
  if (typeof setTimeout !== 'undefined')            pairs.setTimeout = setTimeout;
  if (typeof clearTimeout !== 'undefined')          pairs.clearTimeout = clearTimeout;
  if (typeof setInterval !== 'undefined')           pairs.setInterval = setInterval;
  if (typeof clearInterval !== 'undefined')         pairs.clearInterval = clearInterval;
  if (typeof requestAnimationFrame !== 'undefined')  pairs.requestAnimationFrame = requestAnima...
  if (typeof cancelAnimationFrame !== 'undefined')   pairs.cancelAnimationFrame = cancelAnimati...
```

```
  for (var k in pairs) {
    if (typeof _realGlobal[k] === 'undefined') _realGlobal[k] = pairs[k];
    if (typeof GameGlobal[k] === 'undefined')  GameGlobal[k] = pairs[k];
  }
})();
// ======== 禁用 OffscreenCanvas ========
if (typeof GameGlobal !== 'undefined') {
  GameGlobal.OffscreenCanvas = undefined;
  _realGlobal.OffscreenCanvas = undefined;
}
// ======== WebGL / Canvas2D 上下文构造函数 ========
let _WebGLRenderingContext = {};
try {
  const _tmpCanvas = platform.createCanvas();
  const _tmpGl = _tmpCanvas.getContext('webgl');
  if (_tmpGl) _WebGLRenderingContext = _tmpGl.constructor || {};
} catch (e) { /* 忽略 */ }
let _CanvasRenderingContext2D = {};
try {
  const _tmpCanvas2 = platform.createCanvas();
  const _tmpCtx = _tmpCanvas2.getContext('2d');
  if (_tmpCtx) _CanvasRenderingContext2D = _tmpCtx.constructor || {};
} catch (e) { /* 忽略 */ }
// ======== DOMParser ========
class DOMParser {
  parseFromString() {
    return { documentElement: new Element() };
  }
}
// ======== performance ========
const _performance = typeof performance !== 'undefined' ? performance : {
  now: Date.now.bind(Date),
};
// ======== window 事件系统 ========
const _windowListeners = {};
var _winEvtLogCount = 0;
function _windowAddEventListener(type, handler, options) {
  if (!_windowListeners[type]) _windowListeners[type] = [];
  _windowListeners[type].push(handler);
  _winEvtLogCount++;
  if (_winEvtLogCount <= 20) {
    console.log('[pixi-adapter] globalThis.addEventListener 注册:', type, '(共' + _windowListeners[t...
  }
}
function _windowRemoveEventListener(type, handler) {
  if (!_windowListeners[type]) return;
  const idx = _windowListeners[type].indexOf(handler);
  if (idx !== -1) _windowListeners[type].splice(idx, 1);
}
function _windowDispatchEvent(type, event) {
```

```
  const queue = _windowListeners[type];
  if (queue) {
    const copy = queue.slice();
    copy.forEach(handler => {
      try { handler(event); } catch (e) { console.error('[window event]', type, e); }
    });
  }
}
GameGlobal.__windowDispatchEvent = _windowDispatchEvent;
// ======== 事件构造函数 ========
function _PointerEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }
function _TouchEventCtor(type, opts) { this.type = type; Object.assign(this, opts || {}); }
function _MouseEvent(type, opts) { this.type = type; Object.assign(this, opts || {}); }
// ======== URL / Blob ========
const _URL = {
  createObjectURL: function() { return ''; },
  revokeObjectURL: function() {},
};
function _Blob() {}
// ======== 所有需要挂载的全局属性 ========
const _allGlobals = {
  window: null,         // 下面特殊处理
  document: document,
  navigator: navigator,
  location: location,
  Image: Image,
  Element: Element,
  HTMLCanvasElement: HTMLCanvasElement,
  HTMLImageElement: HTMLImageElement,
  HTMLVideoElement: HTMLVideoElement,
  WebGLRenderingContext: _WebGLRenderingContext,
  CanvasRenderingContext2D: _CanvasRenderingContext2D,
  XMLHttpRequest: XMLHttpRequest,
  DOMParser: DOMParser,
  localStorage: localStorage,
  performance: _performance,
  canvas: canvas,
  ontouchstart: noop,
  addEventListener: _windowAddEventListener,
  removeEventListener: _windowRemoveEventListener,
  self: null,           // 下面特殊处理
  PointerEvent: _PointerEvent,
  TouchEvent: _TouchEventCtor,
  MouseEvent: _MouseEvent,
  URL: _URL,
  Blob: _Blob,
};
if (isDevtools) {
  // ======== 模拟器环境 ========
  // window 已存在（浏览器环境），用 defineProperty 补充/覆盖
```

```
19858  const _win = typeof window !== 'undefined' ? window : GameGlobal;
19859  for (const key in _allGlobals) {
19860    if (key === 'window' || key === 'self') continue;
19861    try {
19862      const desc = Object.getOwnPropertyDescriptor(_win, key);
19863      if (!desc || desc.configurable) {
19864        _origDefineProperty.call(Object, _win, key, { value: _allGlobals[key], configurable: true...
19865      }
19866    } catch (e) { /* 只读属性忽略 */ }
19867  }
19868  // 关键修复：包装 window.addEventListener / removeEventListener
19869  // PixiJS EventSystem 在 globalThis(window) 上注册 pointermove / pointerup，
19870  // 但 adapter 通过 _windowListeners 分发触摸事件--两个系统完全隔离。
19871  // 包装后 handler 会同时进入 _windowListeners，adapter 的 dispatchToWindow 就能触达 PixiJS。
19872  try {
19873    var _nativeWinAdd = _win.addEventListener.bind(_win);
19874    var _nativeWinRemove = _win.removeEventListener.bind(_win);
19875    _win.addEventListener = function(type, handler, options) {
19876      _windowAddEventListener(type, handler, options);
19877      return _nativeWinAdd(type, handler, options);
19878    };
19879    _win.removeEventListener = function(type, handler, options) {
19880      _windowRemoveEventListener(type, handler);
19881      return _nativeWinRemove(type, handler, options);
19882    };
19883    console.log('[pixi-adapter] 模拟器 window.addEventListener 已包装');
19884  } catch (e) {
19885    console.warn('[pixi-adapter] 包装 window.addEventListener 失败:', e);
19886  }
19887  // document 属性补充
19888  try {
19889    for (const key in document) {
19890      const desc = Object.getOwnPropertyDescriptor(_win.document, key);
19891      if (!desc || desc.configurable) {
19892        _origDefineProperty.call(Object, _win.document, key, { value: document[key], configurable...
19893      }
19894    }
19895  } catch (e) { /* 忽略 */ }
19896 } else {
19897  // ======== 真机环境 ========
19898  // 关键：必须同时挂载到 _realGlobal（JS 引擎全局对象）和 GameGlobal（跨文件共享）
19899  // 这样 IIFE bundle 中的自由变量 document、window 等才能正确解析
19900  // window = 全局对象自身（模拟浏览器行为）
19901  _realGlobal.window = _realGlobal;
19902  GameGlobal.window = _realGlobal;
19903  // self = 全局对象自身
19904  _realGlobal.self = _realGlobal;
19905  GameGlobal.self = _realGlobal;
19906  // addEventListener/removeEventListener 必须强制覆盖：
19907  // 微信框架可能内置了无效版本，PixiJS EventSystem 在 self 上注册
```

```
  // pointermove/pointerup 依赖这些函数正确工作
  var _forceOverwrite = new Set(['addEventListener', 'removeEventListener']);
  for (const key in _allGlobals) {
    if (key === 'window' || key === 'self') continue;
    var val = _allGlobals[key];
    var force = _forceOverwrite.has(key);
    // 挂到真正的全局作用域
    if (force || typeof _realGlobal[key] === 'undefined') {
      try { _realGlobal[key] = val; } catch (e) { /* 忽略 */ }
    }
    // 同时挂到 GameGlobal
    if (force || typeof GameGlobal[key] === 'undefined') {
      try { GameGlobal[key] = val; } catch (e) { /* 忽略 */ }
    }
  }
  // 确认事件系统已正确挂载
  console.log('[pixi-adapter] 真机事件系统检查:',
    'globalThis.addEventListener === _windowAddEventListener:', _realGlobal.addEventListener === ...
    ', self.addEventListener === _windowAddEventListener:', (_realGlobal.self && _realGlobal.self...
}
// ======== 全局 canvas ========
// 微信框架可能已将 canvas 设为只读属性，需 try-catch 保护
try { GameGlobal.canvas = canvas; } catch (e) { /* 已由框架设置 */ }
try { _realGlobal.canvas = canvas; } catch (e) { /* 只读属性忽略 */ }
// ======== navigator.userAgent ========
try {
  if (_realGlobal.window && _realGlobal.window.navigator) {
    _realGlobal.window.navigator.userAgent = navigator.userAgent;
  }
} catch (e) { /* 只读属性忽略 */ }
// ======== 注册触摸事件 ========
registerTouchEvents();
console.log('[pixi-adapter] 初始化完成, 平台:', platform.name, ', 环境:', isDevtools ? '模拟器' : '真机');
console.log('[pixi-adapter] _realGlobal === GameGlobal:', _realGlobal === GameGlobal,
  ', typeof document:', typeof _realGlobal.document,
  ', typeof window:', typeof _realGlobal.window);
/**
 * localStorage 模拟，基于平台 Storage API
 */
const platform = require('./platform');
const localStorage = {
  getItem(key) {
    try {
      return platform.getStorageSync(key) || null;
    } catch (e) {
      return null;
    }
  },
  setItem(key, value) {
    try {
```

```
      platform.setStorageSync(key, value);
    } catch (e) {
      console.warn('[localStorage] setItem failed:', key, e);
    }
  },
  removeItem(key) {
    try {
      platform.removeStorageSync(key);
    } catch (e) {
      console.warn('[localStorage] removeItem failed:', key, e);
    }
  },
  clear() {
    console.warn('[localStorage] clear() not implemented in minigame');
  },
  get length() {
    return 0;
  },
  key() {
    return null;
  },
};
module.exports = localStorage;
/**
 * location 对象模拟
 */
const location = {
  href: 'game.js',
  protocol: 'https:',
  host: 'minigame',
  hostname: 'minigame',
  port: '',
  pathname: '/game.js',
  search: '',
  hash: '',
  origin: 'https://minigame',
};
module.exports = location;
/**
 * navigator 对象模拟
 */
const platform = require('./platform');
let _sysInfo;
try {
  _sysInfo = platform.getSystemInfoSync();
} catch (e) {
  _sysInfo = {};
}
const navigator = {
  platform: _sysInfo.platform || 'unknown',
```

```
  language: _sysInfo.language || 'zh_CN',
  appVersion: '5.0 (MiniGame)',
  userAgent: 'Mozilla/5.0 (MiniGame; ' + (_sysInfo.platform || '') + ') PixiJS/7',
  onLine: true,
  maxTouchPoints: 10,
  vendor: '',
  product: '',
  productSub: '',
  hardwareConcurrency: 4,
  // PixiJS 可能会检查 gpu 信息
  gpu: '',
};
module.exports = navigator;
/**
 * 平台抽象层 - 统一微信/抖音小游戏 API
 * 所有 adapter 模块通过此模块调用平台 API，不直接写 wx.xxx 或 tt.xxx
 */
const _isWechat = typeof wx !== 'undefined';
const _isDouyin = typeof tt !== 'undefined';
const _api = _isWechat ? wx : _isDouyin ? tt : null;
if (!_api) {
  console.error('[platform] 未检测到小游戏运行环境（wx/tt）');
}
const platform = {
  createCanvas: () => _api.createCanvas(),
  createImage: () => _api.createImage(),
  getSystemInfoSync: () => _api.getSystemInfoSync(),
  getStorageSync: (key) => _api.getStorageSync(key),
  setStorageSync: (key, data) => _api.setStorageSync(key, data),
  removeStorageSync: (key) => _api.removeStorageSync(key),
  request: (opts) => _api.request(opts),
  connectSocket: (opts) => _api.connectSocket(opts),
  onTouchStart: (cb) => _api.onTouchStart(cb),
  onTouchMove: (cb) => _api.onTouchMove(cb),
  onTouchEnd: (cb) => _api.onTouchEnd(cb),
  onTouchCancel: (cb) => _api.onTouchCancel(cb),
  offTouchStart: (cb) => _api.offTouchStart(cb),
  offTouchMove: (cb) => _api.offTouchMove(cb),
  offTouchEnd: (cb) => _api.offTouchEnd(cb),
  offTouchCancel: (cb) => _api.offTouchCancel(cb),
  createInnerAudioContext: () => _api.createInnerAudioContext(),
  name: _isWechat ? 'wechat' : _isDouyin ? 'douyin' : 'unknown',
  api: _api,
};
module.exports = platform;
/**
 * 工具函数
 */
function noop() {}
module.exports = { noop };
```